

A Smart Parking System with Vehicle Detection and Barrier Control

Nguyen Sy Minh Man, Do Thanh Triet, Tran Dang Khoa, and Bui Dinh Long
 FPT University, Ho Chi Minh Campus, Vietnam
 {syminhman01, thanhtriet1007, khoavainoi2398, buidinhlongwork}@gmail.com

Abstract—This paper presents the design, implementation, and evaluation of a small-scale smart parking prototype that integrates vehicle-presence sensing, image capture, server-side vehicle verification, barrier control, event storage, and web-based monitoring. The system uses one Arduino Uno R3 Master, two Arduino Uno R3 Slaves, two HC-SR04 ultrasonic sensors, two ESP32-CAM modules, one SG90 servo barrier, and a parallel LCD1602 used primarily for local debugging and status display. The ENTRY and EXIT Slaves communicate with the Master through I2C using addresses 0x08 and 0x09, respectively. Each Slave communicates with its corresponding ESP32-CAM through UART at 9600 baud, while the ESP32-CAM captures a JPEG image and uploads it over Wi-Fi to a FastAPI backend. Vehicle verification is performed on the server using the YOLO26l model with a confidence threshold of 0.45 and the allowed classes `car`, `motorcycle`, `bus`, and `truck`. The Arduino Master remains the only module authorized to control the barrier and applies a fail-close policy whenever the camera, network, server, or communication result is invalid or uncertain.

The final experimental dataset contained 146 real ESP32-CAM images of physical vehicle models. YOLO26l produced 66 OPEN decisions and 80 CLOSE decisions, corresponding to a positive-sample vehicle-detection rate of 45.21% and a miss rate of 54.79%. The average server-side inference time was 432.23 ms, with a median of 415.76 ms and a maximum of 837.82 ms. In a separate same-source comparison using the earlier vehicle-model test-case image collection, YOLO26l achieved a positive-sample detection rate of 94.23%, compared with 80.43% for the historical YOLOv8n run, but required substantially higher inference time. Because the main dataset contained only positive vehicle samples, complete accuracy, precision, specificity, false-positive rate, and F1-score could not be calculated. End-to-end gate response time was also not quantitatively measured because synchronized timestamp logging was not implemented across the Arduino Slave, ESP32-CAM, server, Arduino Master, and servo-control stages.

The prototype demonstrates a modular and low-cost architecture in which ultrasonic sensing is used as the initial trigger, server-side computer vision is used as the verification layer, and the Arduino Master performs the final barrier-control decision. The results also identify practical limitations related to low-resolution image qual-

ity, power stability, module dependency, and incomplete end-to-end timing instrumentation.

Index Terms—Internet of Things, smart parking, Arduino Uno, ESP32-CAM, HC-SR04, YOLO26l, vehicle detection, barrier control, FastAPI, web dashboard.

I. INTRODUCTION

A. Background

The continuing growth in private vehicle use has increased the demand for effective parking management in campuses, offices, residential buildings, commercial facilities, and other small- to medium-scale environments. In many of these locations, vehicle entry and exit are still managed manually by security personnel or operators. The operator must observe the approaching vehicle, decide whether the barrier should be opened, record the event when required, and monitor the current parking condition. Although this method is simple to deploy, its effectiveness depends heavily on continuous human attention and consistent record keeping.

Manual parking management can create several operational problems. Vehicle processing may become slow during busy periods, entry and exit records may be incomplete, and administrators may have limited information when reviewing previous parking events. A basic gate mechanism can automate barrier movement, but it does not necessarily provide automatic vehicle detection, verification, event storage, or centralized monitoring. These limitations have motivated the development of Internet of Things (IoT)-based smart parking systems.

IoT-based parking systems combine physical sensing devices, embedded controllers, communication networks, backend services, databases, and monitoring interfaces. Sensors detect changes in the physical environment, microcontrollers process local signals, communication modules transmit data, and software services record or display parking information. Previous studies have demonstrated that IoT-based parking systems can support parking-space monitoring, vehicle-presence detection,

remote status visualization, and centralized parking-data management [1], [2].

A fundamental function of a smart parking system is determining whether an object is present near a parking space or gate. Ultrasonic sensors are commonly used in low-cost parking prototypes because they are inexpensive, widely available, and straightforward to integrate with Arduino-class microcontrollers. An ultrasonic sensor transmits a high-frequency sound pulse and measures the time required for the reflected signal to return. The measured echo duration is then converted into the distance between the sensor and the object.

Studies using IoT sensors and ultrasonic control sensors have shown that this sensing method is suitable for basic parking-space occupancy detection and gate-area monitoring [3], [4]. When the measured distance falls below a configured threshold, the embedded controller can interpret the condition as an object-presence event. This method requires relatively little processing power and can be implemented using low-cost hardware.

However, an ultrasonic sensor only measures distance. It does not determine the semantic identity of the detected object. A person, package, wall, or other obstacle may produce a distance measurement similar to that of a vehicle. Therefore, a system that relies exclusively on ultrasonic sensing may initiate a parking-control cycle without confirming that the detected object is a valid vehicle. It also cannot provide visual evidence that can be inspected after an event.

Camera-based monitoring provides an additional information source. A captured image can be stored as visual evidence and processed using computer-vision or deep-learning techniques. Camera-based parking systems have been used for vehicle identification, parking-event monitoring, and license-plate-related applications [5], [6]. Compared with sensor-only detection, an image contains more information about the detected object. Nevertheless, image-based processing introduces additional requirements involving lighting, camera angle, object scale, network connectivity, computational resources, storage, and processing time.

The ESP32-CAM is appropriate for low-cost IoT prototypes because it combines a camera interface, embedded processing capability, and Wi-Fi communication in a compact module. It can capture a JPEG image and upload it to a backend service without requiring a separate camera controller or network device. Previous research has demonstrated the use of ESP32-CAM modules in touchless parking and automated gate prototypes [7].

Despite these advantages, the ESP32-CAM has limited memory and processing capability compared with a computer or server. Running a computationally demand-

ing object-detection model directly on the ESP32-CAM would create significant implementation constraints. The proposed system therefore separates image acquisition from artificial-intelligence processing. Each ESP32-CAM captures a JPEG image and sends it through HTTP over Wi-Fi to a FastAPI backend. The server then applies the YOLO26l model to determine whether the image contains an allowed vehicle class.

The use of a backend server also allows the system to separate the physical control layer from the computationally intensive image-processing layer. The Arduino controllers remain responsible for deterministic local sensing and barrier-control functions, while the server performs vehicle detection. This architecture reduces the workload of the embedded camera module and allows a larger detection model to be used without placing its full computational requirements on the ESP32-CAM.

The proposed prototype uses a modular embedded architecture. One Arduino Uno R3 Master coordinates two Arduino Uno R3 Slaves. The ENTRY Slave is assigned I2C address 0×08 , while the EXIT Slave is assigned I2C address 0×09 . Each Slave is connected to one HC-SR04 ultrasonic sensor and one ESP32-CAM. The Master communicates with both Slaves through I2C, whereas each Slave exchanges capture commands and detection decisions with its corresponding ESP32-CAM through UART at 9600 baud.

The Arduino Master is the only module permitted to control the SG90 servo barrier. This design prevents different modules from independently issuing conflicting barrier commands. A parallel LCD1602 is connected to the Master and is used primarily as a local debugging and status display during prototype testing. It presents sensor readings, module states, and barrier information to support system integration and troubleshooting. It is not treated as the primary end-user interface.

The system also includes a FastAPI server, SQL Server database components, and a web dashboard. The server receives captured JPEG images, executes the vehicle-detection model, returns a plain-text decision, and records detection information when database logging is enabled. The dashboard is intended to retrieve and present available parking information, such as gate direction, decision results, image paths, confidence values, timestamps, gate states, and occupancy information. These functions are described only to the level supported by the implemented code and available evidence.

B. Problem Statement

Traditional parking management in small- and medium-scale facilities still depends heavily on manual

observation and operator-controlled barriers. This approach requires continuous human involvement and may result in delayed gate operation, inconsistent decisions, incomplete event records, and limited traceability. When an abnormal event occurs, the administrator may not have sufficient information to determine what object triggered the gate, what decision was produced, or whether the barrier responded correctly.

A sensor-only automated gate can reduce manual work, but it does not solve the vehicle-verification problem. An ultrasonic sensor can indicate that an object is located within the configured detection range, but it cannot confirm that the object is a vehicle. If the barrier is opened directly from the sensor threshold, a non-vehicle object may activate the gate. Such a system also lacks a captured image that can be used for event inspection or later analysis.

A camera-only approach also has limitations. Continuous image capture and processing would consume network bandwidth and backend resources even when no object is present. It could also generate unnecessary image records. In addition, camera performance depends on image resolution, object scale, camera angle, lighting conditions, and network stability. A more practical low-cost architecture therefore requires a hybrid detection method in which an inexpensive sensor provides the initial trigger and image-based processing performs a secondary verification step.

The integration of multiple hardware and software components introduces another technical problem. The complete decision path includes ultrasonic detection, Arduino Slave processing, I2C communication, capture-cycle initiation, UART communication, ESP32-CAM image acquisition, Wi-Fi transmission, HTTP processing, YOLO inference, decision return, Master validation, servo command generation, and physical barrier movement. Each stage can fail or produce an invalid result.

For example, the camera may fail to initialize, Wi-Fi may be unavailable, the server address may be invalid, JPEG capture may fail, the HTTP request may time out, the server may return a non-successful response, or UART data may be malformed. A result from a previous capture cycle may also become stale and must not be reused to open the barrier during a new event. Consequently, the system requires explicit control ownership and a fail-safe decision policy.

The proposed system applies a fail-close policy. The barrier begins in the closed position and may open only when the Arduino Master receives a valid OPEN result associated with the active capture cycle. Camera failure, Wi-Fi failure, JPEG capture failure, server-connection failure, HTTP timeout, non-200 response, empty re-

sponse, malformed data, and unknown decisions result in or are treated as CLOSE. This policy prevents uncertain conditions from directly opening the barrier.

Another problem concerns the dependency between the Arduino Master and the two Slave modules. During observed integration tests, the Master operated normally when both Slaves were active and responsive. However, the current implementation stopped normal operation when either the ENTRY or EXIT Slave became inactive. The modular architecture therefore simplifies responsibility separation, but its current implementation does not yet provide fault isolation or degraded operation using only one functional gate module.

Power stability is also critical, particularly for the ESP32-CAM and servo motor. The ESP32-CAM can produce short current-demand peaks during camera initialization, image capture, and Wi-Fi transmission. The servo can also introduce voltage variation when it moves. An unstable supply, excessive wiring resistance, or an unsuitable voltage-conversion path can cause camera resets, communication failures, or inconsistent module behavior. The final design therefore requires a stable regulated 5 V supply, sufficient transient-current capacity, short power wiring, and a common ground between all controllers and peripherals.

A complete smart parking system should also support event review and centralized monitoring. Local barrier movement alone does not provide administrators with structured information about detected objects, server decisions, captured images, gate direction, or parking history. A backend database and dashboard can support these functions, but the available implementation evidence must be distinguished from planned or partially demonstrated capabilities. Unsupported claims regarding complete dashboard reliability, occupancy accuracy, or production-level monitoring must be avoided.

C. Project Objectives

The primary objective of this project is to design, implement, and evaluate a low-cost smart parking prototype that combines ultrasonic vehicle-presence sensing, ESP32-CAM image capture, server-side vehicle detection, embedded communication, and servo-based barrier control.

The specific objectives are as follows:

- 1) To detect objects near the ENTRY and EXIT gate areas using two HC-SR04 ultrasonic sensors.
- 2) To coordinate the gate modules using one Arduino Uno R3 Master and two Arduino Uno R3 Slaves connected through I2C.

- 3) To maintain a consistent module identity in which the ENTRY Slave uses address 0×08 and the EXIT Slave uses address 0×09 .
- 4) To capture event images using two ESP32-CAM modules configured with a 20 MHz camera clock, VGA frame size, JPEG quality value 12, and one frame buffer.
- 5) To transmit JPEG images from the ESP32-CAM modules to a local FastAPI server through HTTP over Wi-Fi.
- 6) To perform server-side vehicle detection using the YOLO26l model with an inference image size of 640 pixels, a confidence threshold of 0.45, and the allowed classes `car`, `motorcycle`, `bus`, and `truck`.
- 7) To transmit the server decision through the ESP32-CAM and Arduino Slave to the Arduino Master.
- 8) To ensure that only the Arduino Master can control the SG90 servo barrier.
- 9) To apply a fail-close policy whenever image capture, network communication, server processing, decision validation, or embedded communication fails.
- 10) To store and display parking-related records through database and dashboard components when those functions are supported by implementation evidence.
- 11) To evaluate the final YOLO26l configuration using verified image-result logs and to report only metrics supported by the available dataset.
- 12) To document observed integration limitations involving image quality, processing time, power stability, module dependency, and incomplete end-to-end timing instrumentation.

D. Scope of the Project

The project is limited to a small-scale laboratory prototype. It uses physical vehicle models rather than full-size vehicles, an SG90 servo rather than an industrial barrier, and low-cost embedded hardware rather than production-grade parking equipment. The prototype demonstrates the logical control process and communication architecture but does not represent a production-ready installation.

The implemented system includes one Master controller, two Slave controllers, two ultrasonic sensors, two ESP32-CAM modules, one shared barrier mechanism, one parallel LCD1602, one local FastAPI vehicle-detection server, supporting SQL Server components, and a Java-based web dashboard. The LCD is used as a debugging and status display rather than a required user interface.

The current system performs vehicle-class detection and returns a binary OPEN or CLOSE decision. License-plate recognition is not implemented and is treated only as a possible future extension. The system does not claim identity-based vehicle authorization, plate matching, user authentication at the gate, payment processing, or production security enforcement.

The main YOLO26l experimental dataset contains only positive images in which each input image includes a physical vehicle model. Therefore, the evaluation supports calculation of a positive-sample detection rate and miss rate, but it does not support complete accuracy, precision, specificity, false-positive rate, or F1-score. Evaluation with negative samples and manually verified ground-truth labels remains outside the completed experiment.

End-to-end gate response time is also outside the available quantitative evaluation because synchronized timestamp logging was not implemented across the Arduino Slave, ESP32-CAM, server, Arduino Master, and servo stages. The report therefore presents YOLO inference time as a server-side measurement rather than treating it as complete system-response time.

The prototype was not evaluated under long-duration operation, unstable Wi-Fi, network congestion, outdoor weather exposure, broad lighting variation, production vehicle distances, simultaneous high-volume ENTRY and EXIT events, or industrial safety requirements. These conditions remain subjects for future testing and system development.

E. Project Contributions

The first contribution of this project is a hybrid vehicle-detection architecture. Ultrasonic sensing is used as the initial event trigger, while ESP32-CAM image capture and server-side YOLO processing provide a second verification layer. This arrangement avoids continuous image processing while providing more evidence than a sensor-only gate mechanism.

The second contribution is a modular controller architecture. Separate ENTRY and EXIT Slaves manage local sensing and camera communication, while the Arduino Master coordinates the complete control process and remains the sole controller of the barrier. The separation of responsibilities provides a clear mapping between sensing, communication, artificial-intelligence processing, and physical actuation.

The third contribution is a fail-close control workflow. The system is designed so that invalid or uncertain conditions cannot directly open the barrier. The Master requires a valid result for the active capture cycle,

while failures in camera initialization, Wi-Fi connectivity, JPEG capture, HTTP communication, server response, or UART communication are treated as closed-gate conditions.

The fourth contribution is the practical use of server-side YOLO26l processing for low-resolution ESP32-CAM images. The project evaluates the selected model using real experimental images of physical vehicle models and reports the resulting decision distribution, positive-sample detection rate, miss rate, confidence values, and inference-time statistics.

The fifth contribution is an evidence-aware evaluation approach. Measured results, observed behavior, configuration parameters, and unmeasured metrics are reported separately. This distinction prevents server inference time from being presented as complete gate-response time and prevents positive-sample detection rate from being described as overall model accuracy.

The remainder of this report presents the hardware and software architecture, communication protocols, system workflow, implementation process, experimental results, observed limitations, and conclusions of the proposed smart parking system.

II. SYSTEM DESIGN

A. Design Requirements

The proposed smart parking system was designed as a small-scale IoT prototype that combines local vehicle-presence sensing, image acquisition, server-side vehicle verification, embedded communication, barrier actuation, and supporting monitoring functions. The design was required to remain low-cost and modular while separating sensing, image processing, and physical barrier control into clearly defined components.

The first design requirement was the use of a hybrid detection process. An HC-SR04 ultrasonic sensor provides the initial object-presence trigger because distance sensing is inexpensive, responsive, and simple to implement using an Arduino-class controller. However, the ultrasonic result is not used as sufficient evidence to open the barrier. Instead, the system requests an image from an ESP32-CAM and sends that image to a backend server for vehicle verification. This design addresses the limitation that an ultrasonic sensor can detect an object but cannot determine whether the object is a valid vehicle [3], [4].

The second requirement was modular separation between the ENTRY and EXIT directions. Each direction contains an independent Arduino Uno R3 Slave, HC-SR04 sensor, and ESP32-CAM. The modules use separate I2C identities so that the Arduino Master can

distinguish the source of a detection event. The ENTRY Slave is assigned address 0×08 , whereas the EXIT Slave is assigned address 0×09 . This mapping must remain consistent across the firmware, circuit diagram, block diagram, server module identifier, test records, and report.

The third requirement was centralized ownership of the barrier. Only the Arduino Uno R3 Master is permitted to generate the control signal for the SG90 servo. The Slave controllers and ESP32-CAM modules provide sensor information and vehicle-detection decisions, but they do not directly operate the barrier. This rule prevents conflicting actuator commands and ensures that the Master can validate the active capture cycle before opening the gate.

The fourth requirement was a fail-close control policy. The barrier starts in the closed position and remains closed whenever the system cannot establish a valid OPEN result. Camera initialization failure, Wi-Fi failure, invalid server configuration, JPEG capture failure, HTTP timeout, non-successful server response, empty response, malformed response, invalid UART data, stale results, and mismatched results must not open the barrier.

The fifth requirement was the use of a stable and electrically appropriate power architecture. The final design uses a regulated 5 V supply rated at approximately 3 A, with all controllers and peripherals connected to a common ground. The ESP32-CAM and servo require stable 5 V branches because image capture, Wi-Fi transmission, and servo movement can create short current-demand variations. The previous 12 V/2 A adapter path is not part of the final power design.

The final requirement was that the system description remain consistent with the implemented prototype. The final system does not include an infrared receiver, infrared remote control, external status LED, or implemented license-plate-recognition function. The LCD1602 is connected through a 4-bit parallel interface and is used primarily as a local debugging and status display during prototype testing.

B. Overall System Architecture

The system uses a modular architecture divided into three principal sections:

- 1) Logic Control and LCD System;
- 2) ENTRY and EXIT Sensor-Camera Modules;
- 3) Local Server, Database, and Dashboard System.

The Logic Control and LCD System contains the Arduino Uno R3 Master, SG90 servo barrier, and parallel LCD1602. The Master coordinates the complete

embedded workflow, polls the two Slave modules, initiates image-capture cycles, validates returned detection decisions, controls the servo barrier, and displays local diagnostic information.

The two Sensor-Camera Modules are structurally symmetrical. Each module contains one Arduino Uno R3 Slave, one HC-SR04 ultrasonic sensor, and one ESP32-CAM. The ENTRY module monitors objects approaching the parking entrance, whereas the EXIT module monitors objects approaching the parking exit. Each Slave measures the local sensor distance, reports its state to the Master, receives a capture request, communicates with its ESP32-CAM, and returns the final vehicle-verification result to the Master.

The Local Server, Database, and Dashboard System contains the FastAPI backend, the YOLO26l vehicle-detection model, SQL Server components, and the Java web dashboard. The ESP32-CAM sends a JPEG image to the FastAPI endpoint through HTTP over Wi-Fi. The backend processes the image using YOLO26l and produces a plain-text OPEN or CLOSE response. When database logging is enabled, the server stores available detection information such as the gate direction, decision, confidence value, detected class, image path, and event timestamp. The dashboard retrieves available records for monitoring and review.

As illustrated in Fig. 1, the architecture separates local real-time control from network-dependent image processing. The Master and Slave controllers continue to manage sensor and barrier states locally, whereas the backend server performs the computationally demanding vehicle-detection task. This separation allows the ESP32-CAM to focus on image acquisition and communication rather than executing the complete AI model locally.

The overall architecture also separates the control path from the monitoring path. The control path is responsible for deciding whether the barrier should open or remain closed. It includes the sensor, Slave, ESP32-CAM, FastAPI server, Master, and servo. The monitoring path records and displays available event information through the database and dashboard. A dashboard failure must not independently open the barrier because the dashboard is not part of the actuator-control authority.

C. Hardware Architecture

Table I summarizes the principal hardware components and their assigned roles.

1) *Arduino Uno R3 Master*: The Arduino Uno R3 Master is the central local controller. Its responsibilities include polling both Slaves, distinguishing ENTRY and EXIT events, starting capture cycles, waiting for results,

validating the active result, controlling the servo, and displaying diagnostic information.

The Master communicates with the Slaves through the I2C bus. It addresses the ENTRY module using 0x08 and the EXIT module using 0x09. The Master polling interval is configured as 1000 ms. This value is a firmware configuration parameter rather than a measured communication latency. An object may therefore wait between almost 0 ms and approximately 1000 ms before the next polling cycle detects the updated Slave state.

The Master is also responsible for maintaining the barrier state. The barrier begins in the closed position. A valid OPEN result for the active capture cycle allows the Master to command the servo to the open position. A CLOSE result, timeout, invalid result, or uncertain state keeps the barrier closed.

The Master does not perform image capture or AI inference. It receives only the final decision propagated through the corresponding Slave. This design keeps the local control logic within the processing capability of the Arduino Uno R3 and separates it from network and image-processing tasks.

2) *Arduino Uno R3 Slaves*: Two Arduino Uno R3 boards operate as directional Slave controllers. The ENTRY Slave is associated with the entrance sensor and camera, while the EXIT Slave is associated with the exit sensor and camera. Their hardware structures are similar, but their addresses and server module names differ.

The Slave reads the HC-SR04 distance measurement and determines whether an object satisfies the configured detection condition. It then exposes its current state to the Master through I2C. When the Master starts a capture cycle, the Slave sends a CAPTURE command to the corresponding ESP32-CAM through UART.

After issuing the capture command, the Slave waits for a plain-text OPEN or CLOSE result. Invalid UART data is ignored until a valid result is received or the configured capture timeout expires. The Slave capture timeout is approximately 6000 ms. If no valid result is available within this interval, the Slave returns or represents a closed decision according to the fail-close policy.

The Slave does not directly control the servo. Its role is limited to local sensing, ESP32-CAM coordination, and communication with the Master. This distinction ensures that an individual directional module cannot independently open the barrier.

3) *HC-SR04 Ultrasonic Sensors*: Each directional module contains one HC-SR04 ultrasonic sensor. The sensor provides the first stage of the hybrid detection process. It transmits an ultrasonic pulse and measures the echo-return duration, which is converted into a distance value.

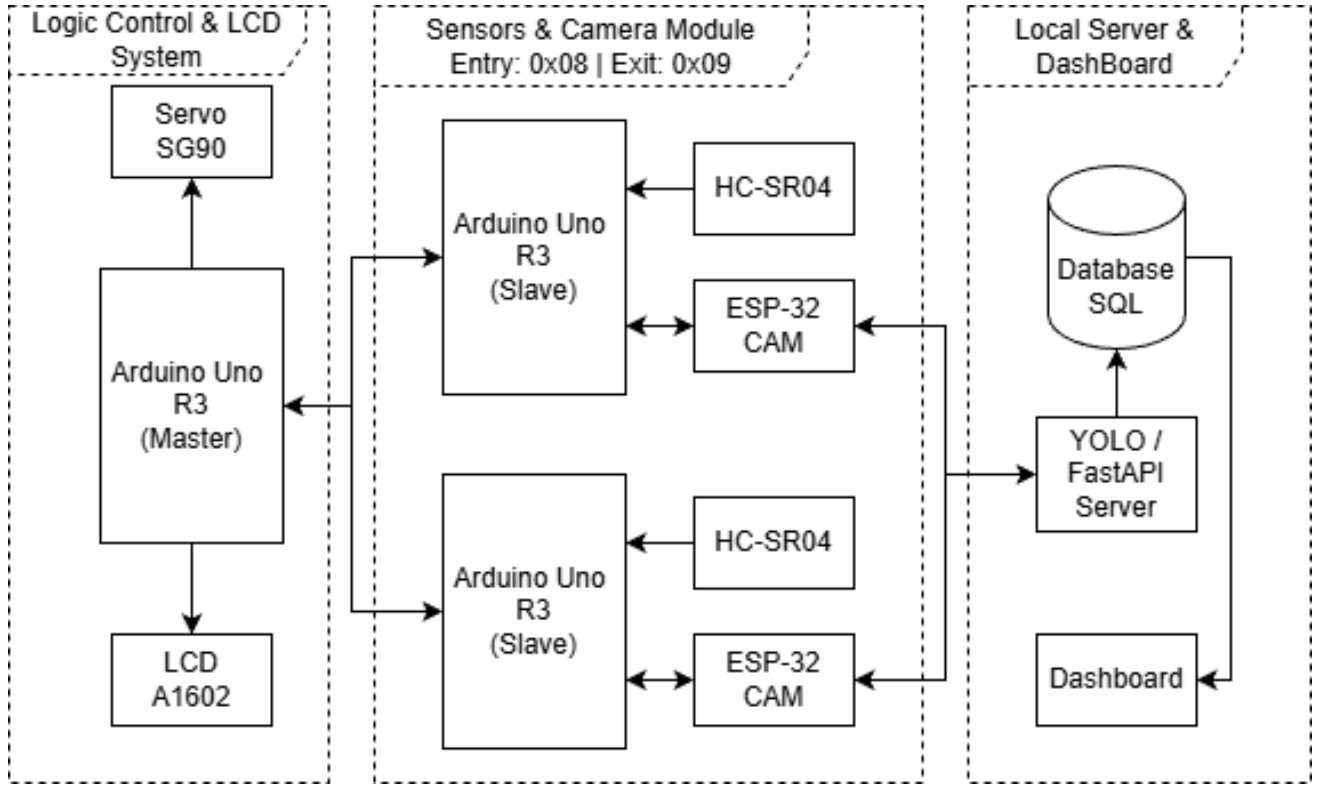


Fig. 1: Block diagram of the proposed smart parking system.

TABLE I: Main Hardware Components of the Proposed System

Component	Quantity	Role in the Proposed System
Arduino Uno R3 Master	1	Coordinates the complete local control process, polls the ENTRY and EXIT Slaves, validates capture results, controls the SG90 barrier, and updates the LCD1602 debugging display.
Arduino Uno R3 Slave	2	Manages one directional sensor-camera module. Each Slave reads an HC-SR04 sensor, communicates with the Master through I2C, and exchanges capture commands and decisions with an ESP32-CAM through UART.
HC-SR04 ultrasonic sensor	2	Measures the distance to an object near the ENTRY or EXIT detection area and provides the initial trigger for a vehicle-verification cycle.
ESP32-CAM	2	Captures a JPEG image for the corresponding ENTRY or EXIT event and uploads it to the FastAPI server through Wi-Fi.
SG90 servo motor	1	Represents the physical parking barrier and changes position according to the PWM signal generated exclusively by the Arduino Master.
LCD1602	1	Operates in 4-bit parallel mode and displays local sensor values, module states, and barrier information during integration and troubleshooting.
Regulated 5 V supply	1	Provides the final system power source with approximately 3 A capacity and a shared ground for all controllers and peripherals.

The ultrasonic sensor is used because it is suitable for low-cost object-presence detection and can be interfaced directly with an Arduino controller [3], [4]. Its output is useful for determining whether an object is located near the gate, but it is not treated as vehicle verification.

When the measured distance enters the configured detection region, the corresponding Slave changes its detection state. The Master observes this state during polling and may start a camera-verification cycle. After the event is processed, the system should wait until the object clears the detection area before allowing another

complete cycle. This requirement prevents repeated capture and barrier actions while the same object remains in front of the sensor.

4) *ESP32-CAM Modules*: The system uses two ESP32-CAM modules: one identified as ENTRY and one identified as EXIT. Each module receives a capture command from its Arduino Slave, captures a JPEG image, uploads the image to the backend server, reads the server response, and sends the resulting decision back to the Slave.

The final camera configuration is:

```

config.xclk_freq_hz = 20000000;
config.frame_size = FRAMESIZE_VGA;
config.jpeg_quality = 12;
config.fb_count = 1;

```

The 20 MHz external camera clock, VGA frame size, JPEG quality value 12, and one-frame-buffer configuration were selected as the final implemented settings. These values are configuration parameters and must not be interpreted as measured performance results.

The ESP32-CAM uses Wi-Fi for HTTP communication with the FastAPI server. It adds the module identity to the request so that the backend can distinguish ENTRY and EXIT events. Camera initialization failure, Wi-Fi failure, capture failure, server-connection failure, HTTP timeout, or an invalid response produces a fail-close result.

The ESP32-CAM is not used to execute the full YOLO model locally. Its role is limited to image acquisition, network transmission, response handling, and UART communication. Server-side processing is more appropriate because the selected YOLO26l model requires substantially greater computational resources than the ESP32-CAM can practically provide.

5) *SG90 Servo Barrier*: The SG90 servo represents the physical barrier mechanism in the laboratory prototype. The Arduino Master generates the PWM control signal and commands the servo to the configured closed or open position.

The servo begins in the closed state during system initialization. It may open only after the Master validates a current OPEN result. The barrier should remain open for at least the configured minimum-open interval, which is approximately 3000 ms. However, the barrier should not close only because this minimum interval has elapsed while an object remains in the barrier zone.

A maximum-open safety timeout may be used to prevent the barrier from remaining open indefinitely if the sensor or state machine fails. The SG90 demonstrates the gate-control concept but is not equivalent to an industrial parking barrier and is not evaluated for full-size vehicle operation or production safety requirements.

6) *Parallel LCD1602*: The LCD1602 is connected directly to the Arduino Master in 4-bit parallel mode. It is not an I2C LCD. Its primary purpose is to support local integration, debugging, and troubleshooting by displaying sensor readings, module states, capture status, and barrier information.

The LCD is not considered the principal user interface. System administrators are expected to use the dashboard for available historical and monitoring information, while the LCD provides immediate local status during proto-

type testing.

Table II shows the agreed LCD1602 connection mapping.

TABLE II: Arduino Master and LCD1602 Interface

Master Pin	LCD Pin	Function
D9	RS	Register-select control
D10	E	Enable control
D5	DB4	Parallel data bit 4
D6	DB5	Parallel data bit 5
D7	DB6	Parallel data bit 6
D8	DB7	Parallel data bit 7
GND	R/W	Fixed write mode
Potentiometer output	VO	Contrast adjustment
5 V	VDD / LED+	Logic and backlight supply
GND	VSS	Ground connection

The LED+ designation in Table II refers to the LCD backlight terminal. It does not indicate the presence of a separate external status LED.

D. Communication Architecture

The system combines I2C, UART, HTTP over Wi-Fi, PWM, parallel LCD signaling, and database communication. Each protocol is assigned to a specific layer according to the processing and communication requirements of that layer.

Table III summarizes the complete interface architecture.

1) *I2C Master-Slave Communication*: I2C is used between the Arduino Master and the two Arduino Slaves because it allows multiple addressed devices to share the same SDA and SCL bus. The Master initiates the communication and polls each Slave according to its assigned address.

The address mapping is fixed as follows:

The Master polling interval is 1000 ms. This interval determines how frequently the Master requests updated information from the Slaves. It does not represent the physical I2C transmission time or the complete system-response delay.

During the observed tests, communication succeeded while both Slaves were active and responsive. However, the current Master implementation stopped normal operation when either Slave became inactive. The design therefore does not yet provide fault isolation or degraded operation using only one active directional module.

2) *UART Communication*: UART is used between each Arduino Slave and its corresponding ESP32-CAM. The implemented baud rate is 9600. The protocol uses simple text commands and decisions to reduce parsing complexity.

TABLE III: Communication and Interface Architecture

Connection	Protocol or Signal	Purpose
Arduino Master ↔ ENTRY Slave	I2C, address 0x08	Transfers sensor state, capture commands, capture-result status, final decisions, and acknowledgement information for the ENTRY module.
Arduino Master ↔ EXIT Slave	I2C, address 0x09	Transfers sensor state, capture commands, capture-result status, final decisions, and acknowledgement information for the EXIT module.
Master D3 → Slave D2	Synchronization signal	Provides the local synchronization signal used by the implemented embedded architecture.
Arduino Slave ↔ ESP32-CAM	UART at 9600 baud	Transfers the CAPTURE command and the OPEN or CLOSE result.
ESP32-CAM ↔ FastAPI server	HTTP over Wi-Fi	Uploads a raw JPEG image and receives a plain-text detection decision.
FastAPI server ↔ SQL Server	Database connection	Stores detection and event information when database logging is enabled.
Dashboard ↔ Backend	API/HTTP	Retrieves available event, gate, image, and occupancy records for monitoring.
Arduino Master → SG90 servo	PWM	Controls the barrier position.
Arduino Master → LCD1602	4-bit parallel interface	Displays local debugging and system-status information.

TABLE IV: Directional Module Identity Mapping

Direction	I2C Address	ESP32-CAM Name
ENTRY	0x08	ENTRY
EXIT	0x09	EXIT

The main command sent from the Slave to the ESP32-CAM is:

CAPTURE

The expected result returned from the ESP32-CAM is one of the following:

OPEN

CLOSE

The Arduino Uno uses 5 V logic, whereas the ESP32-CAM uses 3.3 V logic. Therefore, the Arduino Slave TX signal must pass through a voltage divider before reaching the ESP32-CAM RX input. The ESP32-CAM TX output uses 3.3 V logic and can be read by the Arduino Slave RX input.

The Slave must ignore unknown or malformed UART data until a valid result is received or the timeout expires. It must not interpret an arbitrary string as permission to open the barrier.

3) *HTTP and Wi-Fi Communication*: The ESP32-CAM communicates with the FastAPI backend through HTTP over Wi-Fi. The captured JPEG image is transmitted in an HTTP request to the vehicle-detection endpoint. The request identifies the module as either ENTRY or EXIT.

The ESP32-CAM HTTP timeout is configured at approximately 3500 ms. This value is a code parameter

rather than a measured network delay. If the server cannot be reached, the timeout expires, the HTTP status is not successful, or the returned content is empty or malformed, the result is treated as CLOSE.

HTTP was selected because it provides straightforward integration between the ESP32-CAM and FastAPI backend. It also allows the captured image to be submitted directly to the server using a conventional request-response interaction. The trade-off is that HTTP introduces network and processing overhead compared with purely local embedded communication.

E. Circuit and Interface Design

The complete circuit connects three Arduino Uno R3 boards, two HC-SR04 sensors, two ESP32-CAM modules, one SG90 servo, one parallel LCD1602, a synchronization signal, shared I2C buses, UART connections, and the common 5 V power and ground rails.

As shown in Fig. 2, SDA and SCL are shared between the Master and both Slaves, while each Slave retains its own I2C address. All controllers share a common ground. Each Slave is connected to one ultrasonic sensor and one ESP32-CAM, whereas the servo and LCD1602 are connected only to the Master.

The circuit must preserve the ENTRY and EXIT identity mapping. The ENTRY hardware module, ESP32-CAM configuration, I2C address 0x08, server module value, and dashboard records must all refer to the same physical direction. The corresponding EXIT values must use address 0x09 and module name EXIT.

The final circuit does not include an infrared receiver, infrared remote, or external status LED. Earlier refer-


```

-> ESP32-CAM
-> FastAPI / YOLO26l
-> ESP32-CAM
-> Arduino Slave
-> Arduino Master
-> SG90 barrier

```

A valid OPEN result must belong to the active capture cycle. A stale decision from an earlier event or a mismatched result must not open the barrier. The system must clear or invalidate the previous result when a new cycle begins.

The barrier-control rules are:

- 1) The barrier begins in the closed position.
- 2) Only the Arduino Master controls the SG90 servo.
- 3) Ultrasonic detection alone does not open the barrier.
- 4) The barrier opens only after the Master validates a current OPEN result.
- 5) A CLOSE decision keeps the barrier closed.
- 6) Any timeout, invalid result, or uncertain state is treated as CLOSE.
- 7) The same object must not continuously restart complete capture cycles.
- 8) A new cycle begins only after the previous event is completed and the object clears the detection region.
- 9) The barrier should not close only because the minimum-open period elapsed while an object remains in the barrier area.
- 10) A maximum-open safety timeout may prevent an unlimited open state when a sensor or state transition fails.

This fail-close design does not guarantee production-level safety or security. It establishes a conservative prototype rule in which uncertainty cannot directly authorize barrier opening.

H. Design Rationale

The final architecture reflects the trade-offs identified in the literature review. Ultrasonic sensors provide a low-cost and computationally simple method for initial object detection, but they cannot verify vehicle identity [3], [4]. Camera-based methods provide richer information and visual evidence, but they require additional processing resources and are sensitive to image conditions [5], [7].

The proposed system therefore combines these methods rather than selecting only one. The ultrasonic sensor reduces unnecessary image requests by activating the camera workflow only when an object is present. The ESP32-CAM provides the image, while the backend

server performs the computationally demanding vehicle-detection task. YOLO-based processing was selected for the verification layer because object-detection models can identify vehicle classes within captured images [6].

The Arduino Master-Slave architecture separates the directional sensing modules from the shared barrier controller. This arrangement provides explicit ENTRY and EXIT identities and allows each sensing module to operate with its own sensor and camera. The trade-off is increased communication dependency. The current implementation requires both Slaves to remain active for normal Master operation and therefore requires further development for fault isolation.

The use of HTTP and FastAPI simplifies integration between the ESP32-CAM and the server, while SQL Server and the Java dashboard provide a foundation for centralized event monitoring. Existing smart parking research supports the value of centralized mobile or web-based monitoring [1], [2]. However, monitoring functions are not treated as barrier-control authority and are reported only to the extent supported by the available implementation evidence.

Overall, the selected architecture balances cost, modularity, processing capability, and implementation complexity. It demonstrates how low-cost embedded hardware can be combined with a local AI server to create a vehicle-verification and barrier-control prototype while maintaining clear control ownership and conservative failure behavior.

III. SOFTWARE DESIGN

A. Software Architecture Overview

The software architecture follows the same modular separation as the hardware design. The complete system is divided into five software layers:

- 1) Arduino Master control firmware;
- 2) Arduino Slave sensing and communication firmware;
- 3) ESP32-CAM image-capture and network firmware;
- 4) FastAPI and YOLO26l backend processing;
- 5) SQL Server and Java web-dashboard components.

Each layer performs a specific function and communicates with the adjacent layer using a defined protocol. The Arduino Master controls the system-level state and the SG90 servo barrier. The Arduino Slaves measure ultrasonic distance, expose their states through I2C, and communicate with the ESP32-CAM modules through UART. The ESP32-CAM modules capture JPEG images and submit them to the FastAPI server. The server performs vehicle detection and returns an OPEN or CLOSE

decision. Available detection results can then be stored in SQL Server and retrieved by the Java web dashboard.

The software architecture separates deterministic local control from network-dependent processing. Sensor reading, capture-cycle coordination, and barrier control are implemented on the Arduino controllers. Image processing is assigned to the backend server because the YOLO26l model requires more processing resources than the ESP32-CAM can practically provide. Database and dashboard components operate as monitoring and persistence services and do not directly control the servo barrier.

The complete decision path is:

```
HC-SR04 detection
-> Arduino Slave state
-> Arduino Master polling
-> Capture-cycle command
-> Arduino Slave UART command
-> ESP32-CAM image capture
-> HTTP image upload
-> FastAPI preprocessing
-> YOLO26l inference
-> OPEN/CLOSE server response
-> ESP32-CAM UART response
-> Arduino Slave result state
-> Arduino Master validation
-> SG90 servo command
```

The software applies a fail-close policy throughout this path. A missing, invalid, stale, malformed, or timed-out result is not considered permission to open the barrier.

B. Arduino Master Firmware

1) *Master Responsibilities:* The Arduino Master firmware is responsible for the high-level control logic of the prototype. Its main functions are:

- 1) Initialize the I2C bus, LCD1602, servo, synchronization output, and internal state variables.
- 2) Poll the ENTRY Slave at address 0x08.
- 3) Poll the EXIT Slave at address 0x09.
- 4) Receive the current distance, object-detection state, capture state, and detection result from each Slave.
- 5) Determine whether a new capture cycle should be started.
- 6) Send a capture request to the active Slave.
- 7) Wait for the capture result without using an old result from a previous event.
- 8) Validate whether the returned result belongs to the active capture cycle.
- 9) Open the barrier only when the current result is OPEN.

- 10) Keep or return the barrier to the closed position for CLOSE, timeout, invalid data, or communication failure.
- 11) Display local status information on the parallel LCD1602.

The Master is the only software component that controls the SG90 servo. Neither the Slaves nor the ESP32-CAM modules can directly issue a barrier movement. This ownership rule keeps actuator decisions centralized and prevents conflicting commands.

2) *Master Initialization:* During startup, the Master initializes the servo in the closed position before normal polling begins. This prevents an undefined servo state from leaving the barrier open during boot.

The initialization sequence includes:

- 1) Configure the I2C Master interface.
- 2) Configure the synchronization output connected from Master D3 to Slave D2.
- 3) Initialize the LCD1602 in 4-bit parallel mode.
- 4) Attach the SG90 servo to the configured PWM pin.
- 5) Move the servo to the configured closed angle.
- 6) Reset stored ENTRY and EXIT module states.
- 7) Reset the active capture-cycle state.
- 8) Start periodic Slave polling.

No barrier-opening decision is accepted during initialization. The system must reach a valid idle state before a complete vehicle-verification cycle can begin.

3) *Slave Polling:* The Master polls the ENTRY and EXIT Slaves at a configured interval of 1000 ms. This interval determines how often the Master requests updated module information.

The polling process is logically represented as follows:

- 1) Select the target Slave address.
- 2) Request the current module-state packet.
- 3) Verify that the expected number of bytes or fields is received.
- 4) Decode the distance, object-detection state, capture status, and result.
- 5) Update the stored module state only when the response is valid.
- 6) Treat a missing or malformed response as a communication failure.

The 1000 ms value is a scheduling parameter and not the physical I2C latency. If an object arrives immediately after a polling operation, the Master may not observe the updated state until the next scheduled poll.

4) *Capture-Cycle Management:* A capture cycle begins only when the Master identifies a valid object-presence event and the system is not already processing another event. The Master must prevent the same object from continuously restarting the camera workflow while it remains in the detection zone.

The Master capture-cycle logic contains the following states:

When a cycle begins, the Master clears the previously stored result. A stale `OPEN` value from an earlier event must not authorize the new cycle. The current result must be associated with the active direction and active capture request.

The configured Master capture timeout is approximately 7500–8000 ms. This timeout provides sufficient time for the Slave to issue the UART command, the ESP32-CAM to capture and upload the image, the server to process the request, and the decision to return through the communication chain. If the timeout expires before a valid result is available, the Master applies the fail-close policy.

5) *Barrier-Control Logic*: The barrier starts in the closed position. The Master commands the SG90 servo to the open position only after receiving a valid `OPEN` decision for the current capture cycle.

The barrier-control rules are:

- 1) Ultrasonic detection alone does not open the barrier.
- 2) A `CLOSE` result keeps the barrier closed.
- 3) A capture timeout keeps the barrier closed.
- 4) Invalid or incomplete Slave data keeps the barrier closed.
- 5) A stale result does not open the barrier.
- 6) A result from the wrong direction does not open the barrier.
- 7) Only the Master writes the servo position.

After opening, the barrier remains open for at least the configured minimum-open period of approximately 3000 ms. However, the minimum-open timer is not the only closing condition. The barrier should not close merely because the timer expires while an object remains in the barrier area.

The software should consider both the elapsed open time and the object-clear state. A maximum-open safety timeout may be used to prevent the barrier from remaining open indefinitely if the sensor or state machine fails.

6) *LCD Status Display*: The LCD1602 is used to display local debugging and integration information. Depending on the current state, the Master may display:

- ENTRY and EXIT distance values;
- Slave communication state;
- Active direction;
- Capture status;
- OPEN or CLOSE result;
- Barrier state;
- Timeout or error information.

The LCD output is not treated as a required user-facing function. Its purpose is to support hardware integration,

firmware debugging, and local observation during prototype testing.

C. Arduino Slave Firmware

1) *Slave Responsibilities*: The two Arduino Slave programs use the same general logic but different module identities. The ENTRY Slave uses I2C address `0x08`, and the EXIT Slave uses address `0x09`.

Each Slave is responsible for:

- 1) Reading the HC-SR04 ultrasonic sensor.
- 2) Calculating the measured distance.
- 3) Determining whether an object is within the configured detection region.
- 4) Maintaining the local capture state.
- 5) Responding to Master I2C requests.
- 6) Receiving capture commands from the Master.
- 7) Sending the `CAPTURE` command to the ESP32-CAM through UART.
- 8) Waiting for an `OPEN` or `CLOSE` response.
- 9) Rejecting malformed or unknown UART data.
- 10) Applying a timeout when the camera or server does not return a valid result.
- 11) Exposing the final result to the Master.

The Slave does not directly open or close the barrier.

2) *Ultrasonic Distance Measurement*: The Slave controls the HC-SR04 trigger pin and measures the duration of the echo pulse. The distance is calculated using the propagation speed of sound.

The conceptual calculation is:

$$d = \frac{t \times v}{2} \quad (1)$$

where d is the measured distance, t is the round-trip echo duration, and v is the approximate speed of sound.

The division by two is required because the ultrasonic pulse travels from the sensor to the object and then returns to the sensor.

If no valid echo is received within the configured measurement timeout, the firmware may use an out-of-range value such as 999 cm to represent an unavailable or invalid measurement. This value is not interpreted as object detection.

The object-detection flag becomes active only when the distance satisfies the configured gate-detection condition. The exact distance threshold is a firmware parameter and must not be presented as measured sensor accuracy.

3) *I2C Data Exchange*: Each Slave provides its current state to the Master through the I2C response handler. The transferred state may include:

- measured distance;

TABLE V: Arduino Master Capture-Cycle States

State	Description
IDLE	The system is waiting for a new object-presence event. No capture request is active.
START_CAPTURE	The Master selects the active direction and sends a capture command to the corresponding Slave.
WAIT_RESULT	The Master waits for the Slave to report a final OPEN or CLOSE result.
OPEN_BARRIER	A valid OPEN result has been received for the active cycle, and the Master commands the servo to the open position.
KEEP_CLOSED	The result is CLOSE, invalid, missing, or timed out. The barrier remains closed.
WAIT_OBJECT_CLEAR	The current event has completed, but the object remains within the detection area. A new cycle is blocked until the object clears.

- object-detection flag;
- capture status;
- final decision;
- module status or error state;
- active sequence or cycle information where implemented.

The Slave also receives control commands from the Master. A capture command changes the local state from idle to active and begins the UART interaction with the ESP32-CAM.

The Slave must not continuously restart the camera request while the current object remains present. After returning the result, it waits for acknowledgement or object clearance before returning to a fully idle state.

4) *UART Capture Coordination:* When the Master requests image verification, the Slave sends:

CAPTURE

to the corresponding ESP32-CAM through UART at 9600 baud.

The Slave then waits for one of the following valid responses:

OPEN

CLOSE

The Slave reads incoming UART characters into a bounded buffer. Newline or carriage-return characters may be used to identify the end of a text message. Whitespace should be removed before comparing the result.

The result-processing rules are:

- 1) Accept OPEN only when it exactly matches the expected decision format.
- 2) Accept CLOSE only when it exactly matches the expected decision format.
- 3) Ignore empty strings.
- 4) Ignore malformed or unrelated debug text.
- 5) Prevent buffer overflow by limiting the maximum message length.
- 6) Apply the capture timeout if no valid result is received.

- 7) Convert timeout or invalid final state to CLOSE.

The configured Slave capture timeout is approximately 6000 ms. This is a software parameter and not a measured end-to-end response time.

D. ESP32-CAM Firmware

1) *ESP32-CAM Responsibilities:* Each ESP32-CAM operates as an image-capture and communication gateway between one Arduino Slave and the FastAPI server.

Its responsibilities are:

- 1) Initialize the camera hardware.
- 2) Load stored Wi-Fi, server, and module configuration.
- 3) Connect to the configured Wi-Fi network.
- 4) Receive a CAPTURE command through UART.
- 5) Capture a JPEG image.
- 6) Submit the JPEG image to the FastAPI server.
- 7) Read and validate the server response.
- 8) Return OPEN or CLOSE through UART.
- 9) Return CLOSE for any invalid or uncertain condition.

The ESP32-CAM does not control the SG90 servo and does not directly communicate with the Arduino Master.

2) *Camera Configuration:* The final camera configuration is:

```
config.xclk_freq_hz = 20000000;
config.frame_size = FRAMESIZE_VGA;
config.jpeg_quality = 12;
config.fb_count = 1;
```

The camera uses a 20 MHz external clock, VGA frame size, JPEG quality value 12, and one frame buffer. These values were used as the final stable configuration for the prototype.

The firmware verifies whether camera initialization succeeds before processing capture requests. If initialization fails, the ESP32-CAM must not send OPEN. The result for the request is CLOSE.

3) *Configuration Storage*: The ESP32-CAM firmware can store configuration values in non-volatile memory. The stored values may include:

- Wi-Fi SSID;
- Wi-Fi password;
- FastAPI server URL;
- module identity;
- configuration-valid flag.

The module identity must be either `ENTRY` or `EXIT`. The identity is used when constructing the server request.

A configuration value stored in non-volatile memory may remain available after firmware reflashing unless the storage is explicitly cleared. Therefore, configuration reset must be handled deliberately during testing.

The current system does not implement a heartbeat mechanism. The report must not describe periodic device-heartbeat communication as an implemented function.

4) *Wi-Fi Connection*: The ESP32-CAM connects to the configured wireless network before attempting image upload. The firmware can assign a module-specific hostname, such as:

```
smartparking-entry
smartparking-exit
```

If Wi-Fi is unavailable, the ESP32-CAM returns `CLOSE` to the Slave. The system does not open the barrier using a locally cached or previous server decision.

5) *Image Capture*: After receiving the UART capture command, the ESP32-CAM requests a frame buffer from the camera driver. The frame must contain JPEG data before it is uploaded.

The capture-failure conditions include:

- camera not initialized;
- frame buffer unavailable;
- zero-length image;
- invalid JPEG buffer;
- memory-allocation failure.

When capture fails, the frame buffer is released when necessary, and the decision returned to the Slave is `CLOSE`.

6) *HTTP Upload*: The captured JPEG is sent to the FastAPI detection endpoint. The module name is included as an HTTP query parameter:

```
/detect?module=ENTRY
/detect?module=EXIT
```

The configured ESP32 HTTP timeout is approximately 3500 ms.

A valid server interaction requires:

- 1) a valid server URL;

- 2) active Wi-Fi connectivity;
- 3) successful TCP and HTTP connection;
- 4) HTTP status code 200;
- 5) non-empty response body;
- 6) response text equal to `OPEN` or `CLOSE`.

A negative HTTP client code, timeout, connection failure, non-200 status, empty response, or malformed body produces `CLOSE`.

7) *Server-Response Validation*: The returned server text is trimmed and normalized before comparison.

The response-processing rule is:

```
if response == "OPEN":
    return OPEN
else:
    return CLOSE
```

This means that `CLOSE`, an unknown string, HTML error content, an empty response, or malformed output all result in the closed decision.

The ESP32-CAM sends only the final normalized `OPEN` or `CLOSE` result to the Arduino Slave. Debug logs must not be mixed with the UART decision channel in a way that causes the Slave to interpret log text as a valid result.

E. FastAPI and YOLO26l Server

1) *Backend Responsibilities*: The FastAPI server is responsible for:

- 1) receiving JPEG image requests;
- 2) validating the module name;
- 3) decoding or storing the uploaded image;
- 4) executing the YOLO26l vehicle-detection model;
- 5) filtering detections by allowed class and confidence threshold;
- 6) generating an `OPEN` or `CLOSE` decision;
- 7) saving raw and annotated images where configured;
- 8) recording available detection information in SQL Server;
- 9) returning a plain-text response to the ESP32-CAM.

The selected model is loaded using:

```
from ultralytics import YOLO

model = YOLO("yolo26l.pt")
```

The earlier YOLOv8n model is retained only as a historical baseline and is not the final server model.

2) *Detection Endpoint*: The main detection endpoint accepts the module direction as a query parameter. The server validates that the supplied value is either `ENTRY` or `EXIT`.

The endpoint performs the following operations:

- 1) Read the request body as JPEG bytes.
- 2) Reject an empty image payload.
- 3) Validate the module value.
- 4) Generate event identifiers and file names.
- 5) Save the raw image when configured.
- 6) Decode the image for inference.
- 7) Run YOLO26l with the configured inference parameters.
- 8) Inspect all detected objects.
- 9) Accept detections belonging to the allowed vehicle classes.
- 10) Select the valid detection result according to the decision logic.
- 11) Create the annotated image where enabled.
- 12) Store the result in the database where enabled.
- 13) Return OPEN or CLOSE.

3) *YOLO26l Configuration*: The final inference configuration is summarized in Table VI.

TABLE VI: Final YOLO26l Server Configuration

Parameter	Configured Value
Model file	yolo26l.pt
Inference image size	640 pixels
Confidence threshold	0.45
Allowed class 1	car
Allowed class 2	motorcycle
Allowed class 3	bus
Allowed class 4	truck
Successful decision	OPEN
No valid vehicle detection	CLOSE

An image produces OPEN when the model detects at least one allowed vehicle class with confidence greater than or equal to 0.45. If no allowed class satisfies the decision condition, the result is CLOSE.

The model confidence value is a detection-confidence score and must not be interpreted as overall classification accuracy.

4) *Image Storage*: The server can store two image forms:

- 1) Raw image received from the ESP32-CAM.
- 2) Annotated image containing the accepted bounding box and detection label.

The database stores relative image paths rather than absolute workstation paths. A stored path may identify a file under a structure such as:

```
parking-image/raw/
parking-image/results/annotated/
```

Relative paths improve portability between the back-end and web application. The dashboard should not

depend on a hard-coded path such as a specific Windows user directory.

5) *Backend Failure Handling*: The backend applies a conservative decision policy. Conditions that result in CLOSE include:

- invalid module name;
- empty request body;
- image-decoding failure;
- model inference exception;
- no allowed vehicle class detected;
- confidence below 0.45;
- unexpected processing error.

The server should return a deterministic plain-text decision whenever possible. The ESP32-CAM independently applies fail-close validation if the HTTP request fails or the response is malformed.

F. Database Design

The implemented SQL Server design uses three primary tables:

- 1) parking_occupancy;
- 2) gates;
- 3) detection_events.

The database stores system records for monitoring and later retrieval. It is not part of the direct servo-control authority.

The occupancy table uses constraints to prevent negative values and values greater than the configured capacity. The detection table references the gate table through a foreign key.

Because the current evidence does not provide a complete repeated validation of database consistency, the report does not claim perfect event persistence, occupancy accuracy, or complete transaction reliability.

G. Java Web Dashboard

1) *Dashboard Architecture*: The web dashboard uses a Java Servlet, JSP, DAO, and model structure. The central controller is mapped to:

```
/dashboard
```

The principal software components include:

- DashboardController;
- DashboardDAO;
- DetectionEventDAO;
- GateDAO;
- ParkingOccupancyDAO;
- DashboardOverview;
- DetectionEvent;
- Gate;

TABLE VII: Core SQL Server Tables

Table	Principal Fields	Purpose
parking_occupancy	id, capacity, vehicles_inside, updated_at	Stores the current parking capacity and number of vehicles recorded inside the parking area.
gates	gate_id, is_open, updated_at	Stores the current logical state of the ENTRY and EXIT gates. Gate ID 1 represents ENTRY and gate ID 2 represents EXIT.
detection_events	event_id, gate_id, created_at, decision, confidence, class_name, raw_image_path, annotated_image_path	Stores available server-side vehicle-detection results and related image paths.

- ParkingOccupancy;
- dashboard.jsp.

The controller receives an HTTP GET request, invokes the DAO classes, stores the returned objects in request attributes, and forwards the request to the JSP view.

The conceptual request flow is:

Browser

```

-> DashboardController
-> DAO classes
-> SQL Server
-> Model objects
-> Request attributes
-> dashboard.jsp
-> HTML response

```

2) *Dashboard Information:* According to the available implementation, the dashboard can present information such as:

- current capacity;
- vehicles inside;
- available spaces;
- latest ENTRY detection;
- latest EXIT detection;
- decision result;
- detected class;
- confidence value;
- raw or annotated image;
- gate state;
- event timestamp.

The dashboard is intended as a monitoring interface rather than a direct servo-control interface. The Arduino Master remains responsible for barrier actuation.

3) *Image-Serving Logic:* The image paths stored in SQL Server refer to backend files rather than files located directly inside the Java web root. Therefore, the web application requires an image-serving component.

The image-serving servlet performs the following operations:

- 1) Receive a requested relative image path.
- 2) Normalize the path.

- 3) Reject directory traversal or invalid locations.
- 4) Resolve the path against the configured parking-image base directory.
- 5) Verify that the file exists and is a regular image file.
- 6) Set the appropriate response content type.
- 7) Stream the image bytes to the browser.

This approach avoids using absolute workstation paths directly in JSP code. It also separates the physical backend image directory from the public URL used by the browser.

4) *Dashboard Limitations:* The dashboard and database functions were not supported by sufficient repeated screenshots, timing measurements, and record-validation trials to claim full implementation reliability. Therefore, the report describes their software structure and demonstrated functions but does not claim:

- complete dashboard uptime;
- measured dashboard-update latency;
- perfect database-event completeness;
- verified occupancy accuracy;
- production-level access control;
- complete image-delivery reliability.

H. Software Timeout Configuration

The system uses several timeout and timing parameters. These values coordinate the software workflow but are not complete measured response times.

The complete gate response time includes sensor detection, polling delay, image capture, JPEG upload, inference, response propagation, Master processing, servo command, and servo movement. Since synchronized timestamp logging was not implemented across these stages, the configured values in Table VIII must not be presented as measured end-to-end delay.

I. Failure and Recovery Design

The software handles failures using fail-close behavior.

TABLE VIII: Principal Software Timing Parameters

Parameter	Configured Value	Purpose
Master polling interval	1000 ms	Determines how frequently the Master requests updated state information from both Slaves.
ESP32-CAM HTTP timeout	Approximately 3500 ms	Limits the time spent waiting for the FastAPI request and response.
Slave capture timeout	Approximately 6000 ms	Limits the time the Slave waits for a valid ESP32-CAM UART result.
Master capture timeout	Approximately 7500–8000 ms	Limits the complete capture-result waiting period at the Master level.
Minimum barrier-open time	Approximately 3000 ms	Prevents immediate barrier closing after a valid open command.

TABLE IX: Software Failure Handling

Failure Condition	Software Response	Barrier Effect
Invalid ultrasonic measurement	Represent measurement as invalid or out of range; do not start a valid vehicle cycle.	Remains closed.
Slave I2C response missing	Mark module as unavailable or stop normal processing according to the current Master implementation.	No new valid open decision.
Invalid UART data	Ignore data until a valid message or timeout.	Remains closed.
Camera initialization failure	ESP32-CAM returns or represents CLOSE.	Remains closed.
Wi-Fi connection failure	Skip upload and return CLOSE.	Remains closed.
JPEG capture failure	Release resources and return CLOSE.	Remains closed.
FastAPI unavailable	HTTP request fails or times out.	Remains closed.
Non-200 HTTP response	Reject response.	Remains closed.
Empty or malformed server response	Normalize to CLOSE.	Remains closed.
No allowed YOLO class	Server returns CLOSE.	Remains closed.
Master capture timeout	End active cycle with a closed result.	Remains closed.
Stale result from previous cycle	Clear or reject result during new-cycle validation.	Must not open.

The current implementation does not provide full fault isolation between the two Slaves. When one Slave becomes inactive, the Master was observed to stop normal operation. A future version should maintain separate module-health states and allow one functional direction to continue operating when the other direction fails.

J. Software Design Constraints

The software design is subject to the following constraints:

- 1) Arduino Uno memory and processing capacity limit the complexity of local control logic.
- 2) The ESP32-CAM depends on stable Wi-Fi and stable 5 V power.
- 3) Image-detection performance depends on image quality, lighting, camera angle, and vehicle-model scale.
- 4) HTTP communication adds network-dependent delay.
- 5) The final YOLO26l model requires substantially more server inference time than the historical YOLOv8n model.

- 6) The current Master implementation depends on both Slaves remaining active.
- 7) The database and dashboard have not undergone complete reliability or security evaluation.
- 8) License-plate recognition is not implemented.
- 9) Device heartbeat monitoring is not implemented.
- 10) Complete synchronized end-to-end timing instrumentation is not implemented.

Despite these constraints, the software architecture demonstrates a complete modular path from object detection to image-based decision processing and barrier actuation. It also maintains a clear separation between embedded control, AI processing, data persistence, and dashboard monitoring.

IV. SYSTEM WORKFLOW

A. Workflow Overview

The proposed smart parking system follows a multi-stage workflow that combines local sensing, embedded communication, image capture, server-side vehicle detection, result validation, barrier control, and event monitoring. The complete workflow is divided between

the Arduino Master, the ENTRY and EXIT Arduino Slaves, the ESP32-CAM modules, the FastAPI backend, the YOLO26l model, the SQL Server database, and the Java web dashboard.

The system does not open the barrier immediately after detecting an object with an ultrasonic sensor. Ultrasonic sensing is used only as the initial trigger. A complete verification cycle must be performed before the Arduino Master is permitted to operate the SG90 servo.

The normal decision path is:

```
HC-SR04 ultrasonic sensor
-> Arduino Slave
-> Arduino Master
-> Capture request
-> Arduino Slave
-> ESP32-CAM
-> FastAPI server
-> YOLO26l vehicle detection
-> ESP32-CAM
-> Arduino Slave
-> Arduino Master
-> SG90 servo barrier
```

The monitoring path continues from the backend to the persistence and presentation layers:

```
FastAPI server
-> SQL Server
-> Java DAO layer
-> DashboardController
-> dashboard.jsp
-> Web browser
```

The barrier-control path and dashboard-monitoring path are separated. A dashboard or database failure does not independently authorize the barrier to open. The Arduino Master remains the only component that can control the servo.

B. System Operating States

The overall workflow can be represented using the operating states shown in Table X.

The operating states prevent the same object from repeatedly triggering new image uploads or barrier operations. A complete event must finish before the module becomes eligible for another capture request.

C. Startup Workflow

1) *Arduino Master Startup:* When power is applied, the Arduino Master performs the following sequence:

- 1) Initialize internal variables and module-state structures.

- 2) Configure the I2C bus as the Master.
- 3) Configure the synchronization output connected from Master D3 to Slave D2.
- 4) Initialize the LCD1602 using the 4-bit parallel interface.
- 5) Attach the SG90 servo to the configured PWM pin.
- 6) Move the SG90 servo to the closed-barrier angle.
- 7) Clear any stored ENTRY or EXIT result from a previous runtime.
- 8) Set the active capture direction to none.
- 9) Set the main control state to idle.
- 10) Begin periodic polling of addresses 0x08 and 0x09.

The barrier is closed before normal event processing begins. No stored or uninitialized decision is allowed to open the barrier during startup.

2) *Arduino Slave Startup:* Each Arduino Slave performs the following startup operations:

- 1) Load its fixed directional identity.
- 2) Initialize the HC-SR04 trigger and echo pins.
- 3) Initialize the I2C interface using the assigned Slave address.
- 4) Configure the Master-command receive handler.
- 5) Configure the Master-data request handler.
- 6) Initialize UART communication with the corresponding ESP32-CAM at 9600 baud.
- 7) Reset the measured distance and object-detection state.
- 8) Reset the capture status and previous decision.
- 9) Enter the local idle-monitoring state.

The identity mapping is fixed:

The directional mapping must remain identical in the firmware, block diagram, circuit diagram, HTTP request, server logs, database record, and dashboard.

3) *ESP32-CAM Startup:* Each ESP32-CAM performs the following sequence:

- 1) Initialize serial communication with the Arduino Slave.
- 2) Load the stored Wi-Fi SSID, Wi-Fi password, server URL, and module identity.
- 3) Validate that the module identity is ENTRY or EXIT.
- 4) Configure the camera using the final camera parameters.
- 5) Initialize the camera driver.
- 6) Attempt to connect to the configured Wi-Fi network.
- 7) Start the local configuration or status service where implemented.
- 8) Wait for a UART CAPTURE command.

The final camera configuration is:

TABLE X: Overall System Operating States

State	Main Active Module	Description
SYSTEM_STARTUP	Master, Slaves, ESP32-CAM modules	Controllers initialize their communication interfaces, sensors, camera modules, LCD, and servo. The barrier is placed in the closed position.
IDLE_MONITORING	Master and Slaves	Both Slaves measure ultrasonic distance, while the Master polls the ENTRY and EXIT module states. No capture request is active.
OBJECT_DETECTED	Directional Slave and Master	The corresponding Slave reports that an object has entered the configured detection region.
CAPTURE_REQUESTED	Master and active Slave	The Master selects the active direction and sends a capture-cycle command to the corresponding Slave.
IMAGE_PROCESSING	Active Slave, ESP32-CAM, and backend	The ESP32-CAM captures a JPEG image, uploads it to the server, and waits for the YOLO26l decision.
RESULT_VALIDATION	Active Slave and Master	The OPEN or CLOSE result is propagated to the Master and checked against the active cycle.
BARRIER_OPEN	Master and SG90 servo	The Master opens the barrier after validating a current OPEN result.
BARRIER_CLOSED	Master and SG90 servo	The barrier remains closed after a CLOSE, timeout, communication failure, or invalid result.
WAIT_OBJECT_CLEAR	Master and active Slave	The processed object remains in the detection region. The system prevents a duplicate capture cycle until the object clears.
RETURN_TO_IDLE	Master and Slaves	The previous event is complete, the object has cleared, and the directional module returns to its idle state.

TABLE XI: Startup Identity Mapping

Direction	I2C Address	ESP32-CAM Identity
ENTRY	0x08	ENTRY
EXIT	0x09	EXIT

```

config.xclk_freq_hz = 20000000;
config.frame_size = FRAMESIZE_VGA;
config.jpeg_quality = 12;
config.fb_count = 1;

```

If the camera cannot initialize or Wi-Fi is unavailable, the module does not generate an OPEN result. Any capture request under these conditions is converted to CLOSE.

The current implementation does not use a heartbeat mechanism. Module availability is determined through the actual communication and event-processing behavior rather than periodic heartbeat records.

4) *FastAPI Server Startup*: The backend startup workflow includes:

- 1) Load the required Python libraries and server configuration.
- 2) Load the YOLO26l model from `yolo26l.pt`.
- 3) Prepare the raw-image and annotated-image directories.
- 4) Initialize the database connection configuration where logging is enabled.
- 5) Register the FastAPI routes.
- 6) Start the application server.

7) Wait for incoming HTTP image requests.

The final model is loaded using:

```
from ultralytics import YOLO
```

```
model = YOLO("yolo26l.pt")
```

The earlier YOLOv8n model is not used as the final detection model. Its results are retained only for historical comparison.

D. Idle Monitoring Workflow

After startup, the embedded system enters the idle-monitoring state. Both Slaves repeatedly measure their local ultrasonic distances. The Master periodically polls both Slaves at the configured 1000 ms interval.

The idle-monitoring sequence is:

- 1) The ENTRY Slave triggers its HC-SR04 sensor.
- 2) The ENTRY Slave measures the echo duration and calculates the distance.
- 3) The EXIT Slave performs the same measurement independently.
- 4) Each Slave updates its local object-detection flag.
- 5) The Master polls the ENTRY Slave at 0x08.
- 6) The Master validates and stores the returned ENTRY state.
- 7) The Master polls the EXIT Slave at 0x09.
- 8) The Master validates and stores the returned EXIT state.

- 9) The Master updates the LCD with available sensor and module information.
- 10) If neither module reports a new object, the system remains idle.

An unavailable ultrasonic echo may be represented by an out-of-range value such as 999 cm. This value indicates that no valid nearby object measurement is available and does not start a capture cycle.

The Master polling interval is a scheduling configuration. It must not be described as the I2C transmission delay or complete system-response time.

E. Object-Detection Workflow

When a valid ultrasonic distance falls inside the configured detection region, the corresponding Slave marks the object-detection condition as active.

The detection workflow is:

- 1) An object enters the HC-SR04 detection region.
- 2) The sensor echo duration changes.
- 3) The Slave calculates a distance below the configured detection threshold.
- 4) The Slave sets its detection flag.
- 5) The Master observes the updated flag during the next polling cycle.
- 6) The Master verifies that no conflicting capture cycle is already active.
- 7) The Master selects the corresponding direction.
- 8) The Master clears the previously stored result for that direction.
- 9) The Master starts a new capture cycle.

The distance threshold is a firmware configuration value. The report does not treat this threshold as measured sensor accuracy.

Ultrasonic detection alone cannot open the barrier. Its purpose is to identify when the image-verification workflow should begin.

F. ENTRY Workflow

The ENTRY process begins when the ENTRY Slave at address `0x08` reports a new object.

The complete ENTRY workflow is:

- 1) The ENTRY HC-SR04 detects an object near the entrance.
- 2) The ENTRY Slave updates its local detection state.
- 3) The Master polls address `0x08` and receives the ENTRY state.
- 4) The Master verifies that the system is ready for a new cycle.
- 5) The Master sets the active direction to ENTRY.
- 6) The Master clears any previous ENTRY decision.

- 7) The Master sends a capture command to the ENTRY Slave.
- 8) The ENTRY Slave changes its capture status from idle to active.
- 9) The ENTRY Slave sends `CAPTURE` to the ENTRY ESP32-CAM through UART.
- 10) The ENTRY ESP32-CAM captures a JPEG frame.
- 11) The ESP32-CAM sends the image to:

`/detect?module=ENTRY`

The workflow then continues:

- 12) The FastAPI server validates the ENTRY module value.
- 13) The server decodes the JPEG image.
- 14) YOLO26l performs vehicle detection.
- 15) The server checks whether an allowed vehicle class satisfies the confidence threshold.
- 16) The server produces `OPEN` or `CLOSE`.
- 17) The server stores the available ENTRY event data where database logging is enabled.
- 18) The server returns the plain-text result to the ENTRY ESP32-CAM.
- 19) The ESP32-CAM normalizes and validates the response.
- 20) The ESP32-CAM sends `OPEN` or `CLOSE` to the ENTRY Slave.
- 21) The ENTRY Slave stores the result and exposes it to the Master.
- 22) The Master polls or requests the final ENTRY result.
- 23) The Master verifies that the result belongs to the active ENTRY cycle.
- 24) If the result is `OPEN`, the Master opens the barrier.
- 25) If the result is `CLOSE`, invalid, missing, or timed out, the barrier remains closed.
- 26) The system waits for the object to clear before resetting the ENTRY module.

An ENTRY `OPEN` result indicates only that YOLO26l detected at least one allowed vehicle class above the configured threshold. It does not represent license-plate authorization, driver authentication, or payment verification.

G. EXIT Workflow

The EXIT workflow uses the same general processing stages but uses the EXIT Slave and EXIT ESP32-CAM.

The complete EXIT workflow is:

- 1) The EXIT HC-SR04 detects an object near the exit.
- 2) The EXIT Slave updates its local detection state.
- 3) The Master polls address `0x09` and receives the EXIT state.

- 4) The Master confirms that no other capture cycle is active.
- 5) The Master sets the active direction to EXIT.
- 6) The Master clears any previous EXIT decision.
- 7) The Master sends a capture command to the EXIT Slave.
- 8) The EXIT Slave sends CAPTURE through UART.
- 9) The EXIT ESP32-CAM captures a JPEG image.
- 10) The ESP32-CAM sends the image to:

/detect?module=EXIT

The workflow continues:

- 11) The FastAPI server validates the EXIT module value.
- 12) YOLO26l processes the image.
- 13) The backend produces OPEN or CLOSE.
- 14) Available EXIT event information is stored in the database.
- 15) The result is returned to the EXIT ESP32-CAM.
- 16) The EXIT ESP32-CAM sends the normalized result to the EXIT Slave.
- 17) The EXIT Slave exposes the result through I2C.
- 18) The Master validates the active EXIT result.
- 19) A valid OPEN causes the Master to open the barrier.
- 20) A CLOSE, timeout, or invalid state keeps the barrier closed.
- 21) The system waits for the EXIT object to clear before returning to idle.

The ENTRY and EXIT workflows share the same server model and decision rule. Their direction is distinguished using the I2C address, ESP32-CAM module name, HTTP query parameter, gate identifier, and stored database record.

H. Arduino Slave Capture Workflow

After receiving a capture request from the Master, the active Slave performs the following local workflow:

- 1) Verify that the current capture state is idle.
- 2) Clear the previous UART result.
- 3) Mark the capture status as busy.
- 4) Record the capture-start time using the local timer.
- 5) Send the CAPTURE message through UART.
- 6) Repeatedly check the UART receive buffer.
- 7) Assemble incoming characters into a bounded message.
- 8) Remove carriage-return, newline, and surrounding whitespace.
- 9) Compare the message with OPEN and CLOSE.
- 10) Accept only an exact valid decision.

- 11) Ignore unrelated debug text or malformed messages.
- 12) Stop waiting when a valid result is received.
- 13) Stop waiting when the approximately 6000 ms capture timeout expires.
- 14) Convert timeout or unresolved state to CLOSE.
- 15) Expose the completed result to the Arduino Master.

The Slave must not reuse a previous UART result. The result buffer and capture status are reset at the beginning of each cycle.

The approximately 6000 ms timeout is a code configuration value. It does not represent a measured average capture duration.

I. ESP32-CAM Capture and Upload Workflow

The ESP32-CAM begins processing only after receiving the UART CAPTURE command.

The image-processing workflow is:

- 1) Read and normalize the UART command.
- 2) Verify that the command is exactly CAPTURE.
- 3) Verify that the camera initialized successfully.
- 4) Verify that the stored server URL and module identity are valid.
- 5) Verify that Wi-Fi is connected.
- 6) Request a frame buffer from the camera driver.
- 7) Verify that the frame contains valid JPEG data.
- 8) Create the HTTP request.
- 9) Append the module query parameter.
- 10) Set the request content type for JPEG data.
- 11) Apply the configured HTTP timeout of approximately 3500 ms.
- 12) Send the JPEG bytes to the FastAPI server.
- 13) Receive the HTTP status code and response body.
- 14) Release the camera frame buffer.
- 15) Validate that the HTTP status code is 200.
- 16) Trim and normalize the response text.
- 17) Return OPEN only when the response exactly matches OPEN.
- 18) Return CLOSE for all other conditions.

The ESP32-CAM applies fail-close behavior independently of the backend. Even when the backend is intended to return a valid decision, an interrupted HTTP connection or malformed body cannot produce an open result.

J. FastAPI and YOLO26l Workflow

The FastAPI detection endpoint processes each image request using the following sequence:

- 1) Receive the HTTP request.
- 2) Read the module query parameter.

- 3) Verify that the module is ENTRY or EXIT.
- 4) Read the JPEG request body.
- 5) Reject an empty image payload.
- 6) Generate a timestamp or event identifier.
- 7) Create the raw-image file path.
- 8) Save the raw image where configured.
- 9) Decode the JPEG bytes into an image representation.
- 10) Run YOLO26l using an inference size of 640 pixels.
- 11) Inspect the generated detection boxes.
- 12) Filter detections using the 0.45 confidence threshold.
- 13) Retain only the allowed classes:

```
car
motorcycle
bus
truck
```

The workflow then continues:

- 14) Determine whether at least one allowed detection remains.
- 15) Set the decision to OPEN when a valid vehicle detection exists.
- 16) Set the decision to CLOSE when no valid vehicle detection exists.
- 17) Generate an annotated image where enabled.
- 18) Save the raw and annotated relative paths.
- 19) Insert the available detection record into SQL Server where logging is enabled.
- 20) Return the plain-text decision to the ESP32-CAM.

The decision rule is:

$$D = \begin{cases} \text{OPEN,} & \text{if an allowed class has } c \geq 0.45, \\ \text{CLOSE,} & \text{otherwise,} \end{cases} \quad (2)$$

where D is the server decision and c is the model confidence score.

The confidence score indicates the model confidence for the accepted detection. It is not equivalent to overall model accuracy.

K. Barrier-Opening Workflow

The Arduino Master opens the barrier only after all of the following conditions are satisfied:

- 1) A valid directional detection event is active.
- 2) The Master initiated the current capture cycle.
- 3) The corresponding Slave completed the capture workflow.
- 4) The returned result is valid.
- 5) The result belongs to the active direction.

- 6) The result is not from a previous cycle.
- 7) The decision is exactly OPEN.
- 8) The Master capture timeout has not expired.

When these conditions are satisfied, the Master:

- 1) Changes the barrier state to opening.
- 2) Commands the SG90 servo to the configured open angle.
- 3) Records the barrier-open start time.
- 4) Updates the LCD status.
- 5) Maintains the barrier in the open state for at least the configured minimum-open period.
- 6) Monitors whether the object remains in the relevant detection region.

The minimum-open duration is approximately 3000 ms. This value is a control parameter and not a measured vehicle-passage time.

L. Barrier-Closing Workflow

The barrier-closing workflow considers both time and object presence.

The normal closing sequence is:

- 1) The barrier has remained open for at least the minimum-open period.
- 2) The associated object clears the detection region.
- 3) The Master confirms that no valid passage event remains active.
- 4) The Master commands the servo to the closed position.
- 5) The barrier state changes to closed.
- 6) The Master clears the completed capture-cycle state.
- 7) The corresponding Slave resets its completed result.
- 8) The system returns to idle monitoring.

The barrier should not close solely because the minimum timer expires while an object remains in the barrier area. Closing under that condition could cause the prototype barrier to move before the object has cleared.

A maximum-open safety timeout may be included to prevent an indefinitely open state when the sensor remains active because of a failure. Such a timeout is a protective fallback and not the preferred normal closing condition.

M. CLOSE Decision Workflow

When the server returns CLOSE, the result propagates through the same communication path:

```
FastAPI
-> ESP32-CAM
-> Arduino Slave
```

-> Arduino Master

The Master then:

- 1) Verifies that the result belongs to the active cycle.
- 2) Keeps the servo in the closed position.
- 3) Updates the LCD with the closed decision.
- 4) Marks the capture cycle as completed without barrier opening.
- 5) Waits for the object to clear.
- 6) Resets the directional event after object clearance.

A CLOSE result indicates that no allowed class satisfied the server decision rule. It does not necessarily prove that the image contained no physical vehicle. The final experiment demonstrated that some positive vehicle-model images still produced CLOSE decisions.

N. Timeout Workflow

The system uses layered timeout handling.

The nested timeout values provide additional protection at each layer. A failure that is not resolved by the ESP32-CAM is eventually handled by the Slave or Master.

These timeout values must not be added together and presented as measured end-to-end latency. They represent upper control limits configured in different modules.

O. Failure Workflows

1) *Camera Initialization Failure:* When camera initialization fails:

- 1) The ESP32-CAM records the camera as unavailable.
- 2) A later CAPTURE request cannot obtain a valid frame.
- 3) The ESP32-CAM returns CLOSE.
- 4) The Slave reports the closed result.
- 5) The Master keeps the barrier closed.

2) *Wi-Fi Failure:* When Wi-Fi is unavailable:

- 1) The ESP32-CAM cannot establish a server connection.
- 2) The image upload is skipped or fails.
- 3) No previous server result is reused.
- 4) The ESP32-CAM returns CLOSE.
- 5) The barrier remains closed.

3) *JPEG Capture Failure:* When the camera frame buffer is unavailable or invalid:

- 1) The ESP32-CAM detects the missing or zero-length JPEG.
- 2) The HTTP request is not treated as successful.
- 3) Allocated resources are released where necessary.
- 4) The result becomes CLOSE.
- 5) The barrier remains closed.

4) *FastAPI Server Failure:* When the FastAPI service is unavailable:

- 1) The ESP32-CAM attempts to send the HTTP request.
- 2) The connection fails or the timeout expires.
- 3) The ESP32-CAM rejects the transaction.
- 4) The ESP32-CAM sends CLOSE to the Slave.
- 5) The Master keeps the barrier closed.

5) *Malformed Server Response:* When the HTTP response is empty, unknown, or malformed:

- 1) The ESP32-CAM trims and normalizes the response.
- 2) The response does not exactly match OPEN.
- 3) The ESP32-CAM converts the result to CLOSE.
- 4) The Slave and Master process the closed decision.

6) *Invalid UART Data:* When the Slave receives debug output or malformed UART content:

- 1) The message is compared with the accepted decision strings.
- 2) Unknown content is ignored.
- 3) The Slave continues waiting while the capture timeout remains active.
- 4) If no valid message is received, the timeout generates CLOSE.

7) *Inactive Slave:* The observed behavior of the current implementation is:

- When both Slaves are active, Master-Slave communication succeeds during the observed normal tests.
- When the ENTRY Slave becomes inactive, the Master stops normal operation.
- When the EXIT Slave becomes inactive, the Master stops normal operation.
- When both Slaves become inactive, the Master stops normal operation.

The current workflow therefore does not support continued operation using only one healthy directional module. Fault isolation is identified as a future improvement.

P. Object-Clear and Cycle-Reset Workflow

After an OPEN or CLOSE result has been processed, the system enters the object-clear stage.

The reset sequence is:

- 1) Mark the capture cycle as completed.
- 2) Prevent another capture request for the same active direction.
- 3) Continue measuring the ultrasonic distance.
- 4) Wait until the object no longer satisfies the detection condition.
- 5) Apply any configured clear-state stabilization or debounce logic.

TABLE XII: Timeout Workflow by Processing Layer

Layer	Configured Limit	Timeout Response
ESP32-CAM HTTP request	Approximately 3500 ms	Abort the request, reject the incomplete response, and return CLOSE through UART.
Arduino Slave capture wait	Approximately 6000 ms	Stop waiting for UART data, store CLOSE, and expose the completed result to the Master.
Arduino Master result wait	Approximately 7500–8000 ms	End the active capture cycle with a closed decision and keep the barrier closed.

- 6) Reset the local detection flag.
- 7) Clear the completed result.
- 8) Return the Slave capture status to idle.
- 9) Clear the Master's active direction.
- 10) Return the complete system to idle monitoring.

This workflow prevents repeated images and repeated servo cycles while the same vehicle model remains stationary in front of the sensor.

Q. Database Recording Workflow

When database logging is enabled, the FastAPI server records the result after processing the image.

The database workflow is:

- 1) Convert the module name to the corresponding gate identifier.
- 2) Generate or retrieve the event timestamp.
- 3) Store the decision as OPEN or CLOSE.
- 4) Store the accepted confidence value where available.
- 5) Store the detected class where available.
- 6) Store the raw-image relative path.
- 7) Store the annotated-image relative path.
- 8) Insert the record into `detection_events`.
- 9) Update the corresponding gate state where implemented.
- 10) Update parking occupancy where the implemented business rule permits.

The current database schema uses:

- gate ID 1 for ENTRY;
- gate ID 2 for EXIT.

A database insert failure should not change an uncertain detection into an open decision. The monitoring and persistence layer is separate from the actuator-authority layer.

R. Dashboard Retrieval Workflow

The Java web dashboard retrieves available records through the controller and DAO layers.

The dashboard workflow is:

- 1) The browser requests the `/dashboard` URL.
- 2) The servlet container invokes `DashboardController`.

- 3) The controller calls `DashboardDAO`.
- 4) The controller calls `DetectionEventDAO`.
- 5) The controller calls `GateDAO`.
- 6) The controller calls `ParkingOccupancyDAO`.
- 7) Each DAO executes the required SQL query.
- 8) The results are mapped to Java model objects.
- 9) The controller stores the objects as request attributes.
- 10) The request is forwarded to `dashboard.jsp`.
- 11) JSP and JSTL render the available overview and event information.
- 12) The browser requests a raw or annotated image through the image-serving servlet.
- 13) The servlet resolves the relative path and streams the image to the browser.

The dashboard can present the latest available ENTRY and EXIT detections, gate state, occupancy values, decisions, detected classes, confidence values, timestamps, and images where those records exist.

The dashboard workflow does not send a direct servo command. The Master remains the sole barrier controller.

S. Simultaneous ENTRY and EXIT Events

The architecture contains independent ENTRY and EXIT sensing modules but only one shared Arduino Master and one SG90 servo barrier. Therefore, simultaneous detections require controlled scheduling.

The safe workflow is:

- 1) The Master detects that one capture cycle is already active.
- 2) The second direction is not allowed to overwrite the current active direction.
- 3) The active cycle completes or times out.
- 4) The associated object-clear workflow begins.
- 5) The Master reevaluates the pending directional states.
- 6) A new cycle begins only when the system returns to an eligible state.

The report does not claim that simultaneous ENTRY and EXIT events were quantitatively evaluated. Priority behavior should not be described as a measured system property unless supported by direct test evidence.

T. Timing Boundaries

The complete end-to-end workflow contains multiple timing components:

$$T_{\text{total}} = T_{\text{sensor}} + T_{\text{poll}} + T_{\text{capture}} + T_{\text{upload}} \quad (3)$$

$$+ T_{\text{inference}} + T_{\text{return}} + T_{\text{embedded}} + T_{\text{servo}}, \quad (4)$$

where:

- T_{sensor} is the ultrasonic detection time;
- T_{poll} is the waiting time until the Master observes the event;
- T_{capture} is the JPEG acquisition time;
- T_{upload} is the HTTP image-upload time;
- $T_{\text{inference}}$ is the YOLO26l processing time;
- T_{return} is the HTTP response-return time;
- T_{embedded} is the UART, I2C, and Master processing time;
- T_{servo} is the physical servo-movement time.

The current prototype did not include synchronized timestamp logging across these stages. Therefore, the report uses the following limitation statement:

End-to-end gate response time was not quantitatively measured because the current prototype did not include synchronized timestamp logging across the Arduino Slave, ESP32-CAM, server, and Arduino Master. Therefore, only the YOLO26l inference time obtained from the server logs is reported.

The measured YOLO inference time is only one component of T_{total} .

U. Master Flowchart

The Arduino Master flowchart represents initialization, Slave polling, event selection, capture-cycle initiation, result validation, barrier control, object clearance, and return to idle.

As shown in Fig. 3, the Master does not directly communicate with the backend. It receives the final decision through the active Slave and opens the barrier only after validating the current result.

V. Slave Flowchart

The Arduino Slave flowchart represents ultrasonic measurement, I2C state exchange, capture-command processing, UART communication, timeout handling, result storage, and object-clear reset.

The ENTRY and EXIT Slaves use the same general workflow but have different I2C addresses and ESP32-CAM identities.

W. ESP32-CAM Flowchart

The ESP32-CAM flowchart represents camera initialization, configuration loading, Wi-Fi connection, UART command reception, JPEG capture, HTTP upload, response validation, and fail-close handling.

Every error branch in Fig. 5 leads to CLOSE. Only an exact valid OPEN server response is forwarded as permission for the next control stage.

X. YOLO Server Flowchart

The server flowchart represents HTTP request validation, image decoding, YOLO26l inference, class filtering, decision generation, image storage, database logging, and response delivery.

The decision is OPEN only when at least one accepted vehicle detection satisfies the configured class and confidence conditions. Otherwise, the decision is CLOSE.

Y. Workflow Summary

The complete workflow combines a low-cost sensor trigger with a camera-based vehicle-verification stage. The HC-SR04 sensor determines when a potential parking event occurs, but the Arduino Master does not open the barrier directly from that signal. The corresponding ESP32-CAM captures an image and sends it to the FastAPI server, where YOLO26l applies the configured vehicle-class and confidence rules.

The result returns through the ESP32-CAM and Arduino Slave to the Arduino Master. The Master then performs the final decision validation and controls the SG90 servo. Invalid, unavailable, malformed, stale, or timed-out results keep the barrier closed.

After the event, the system waits for the object to clear before resetting the active module. This prevents repeated cycles for the same object. Database and dashboard functions provide supporting event storage and monitoring but do not replace the embedded barrier-control authority.

V. IMPLEMENTATION

A. Implementation Overview

The proposed smart parking system was implemented as an integrated laboratory prototype consisting of embedded controllers, ultrasonic sensors, camera modules, a servo barrier, a local debugging display, a vehicle-detection server, an SQL Server database, and a Java web dashboard. The implementation followed the modular architecture described in Sections II, III, and IV.

The embedded implementation contains one Arduino Uno R3 Master and two Arduino Uno R3 Slaves. The

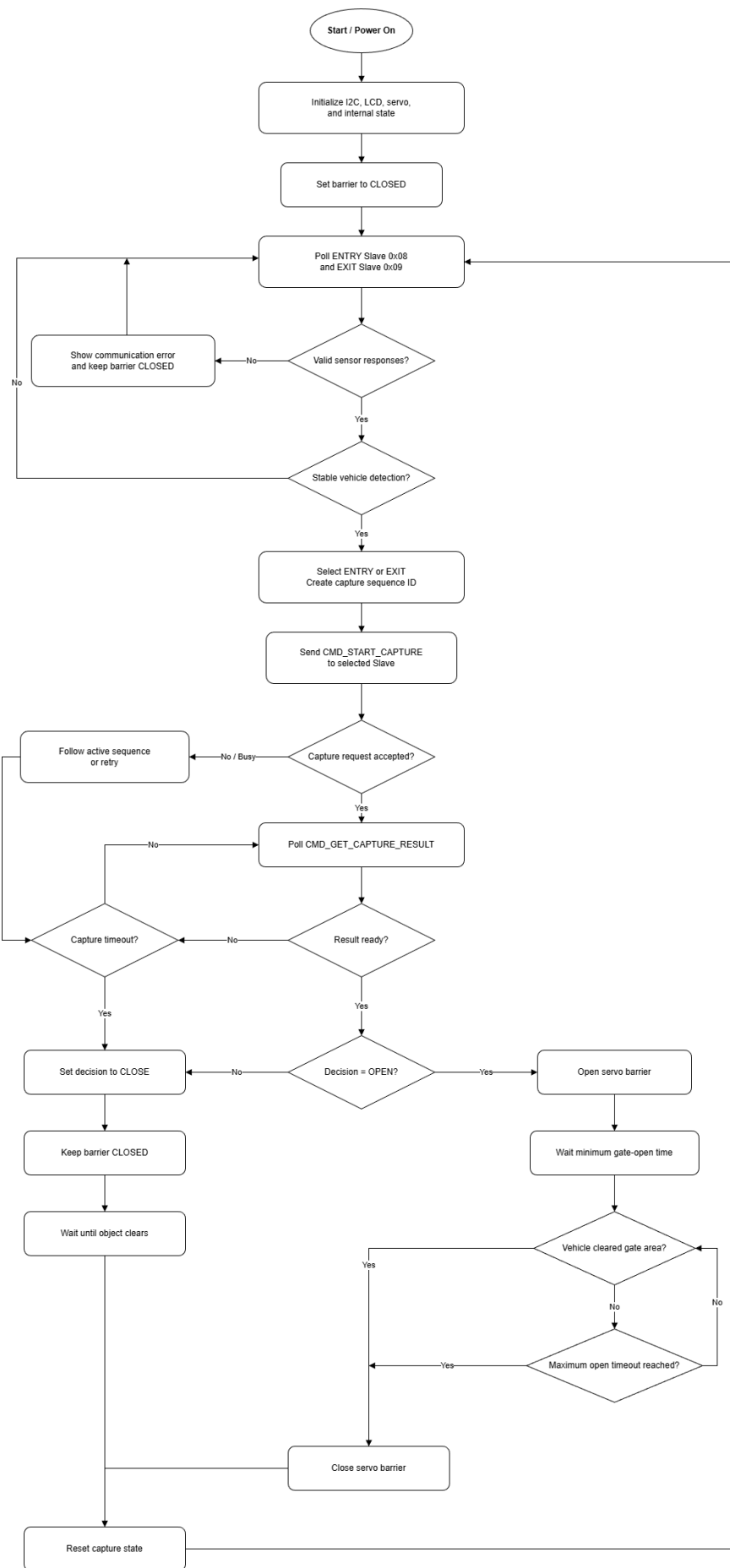


Fig. 3: Workflow of the Arduino Master control firmware.

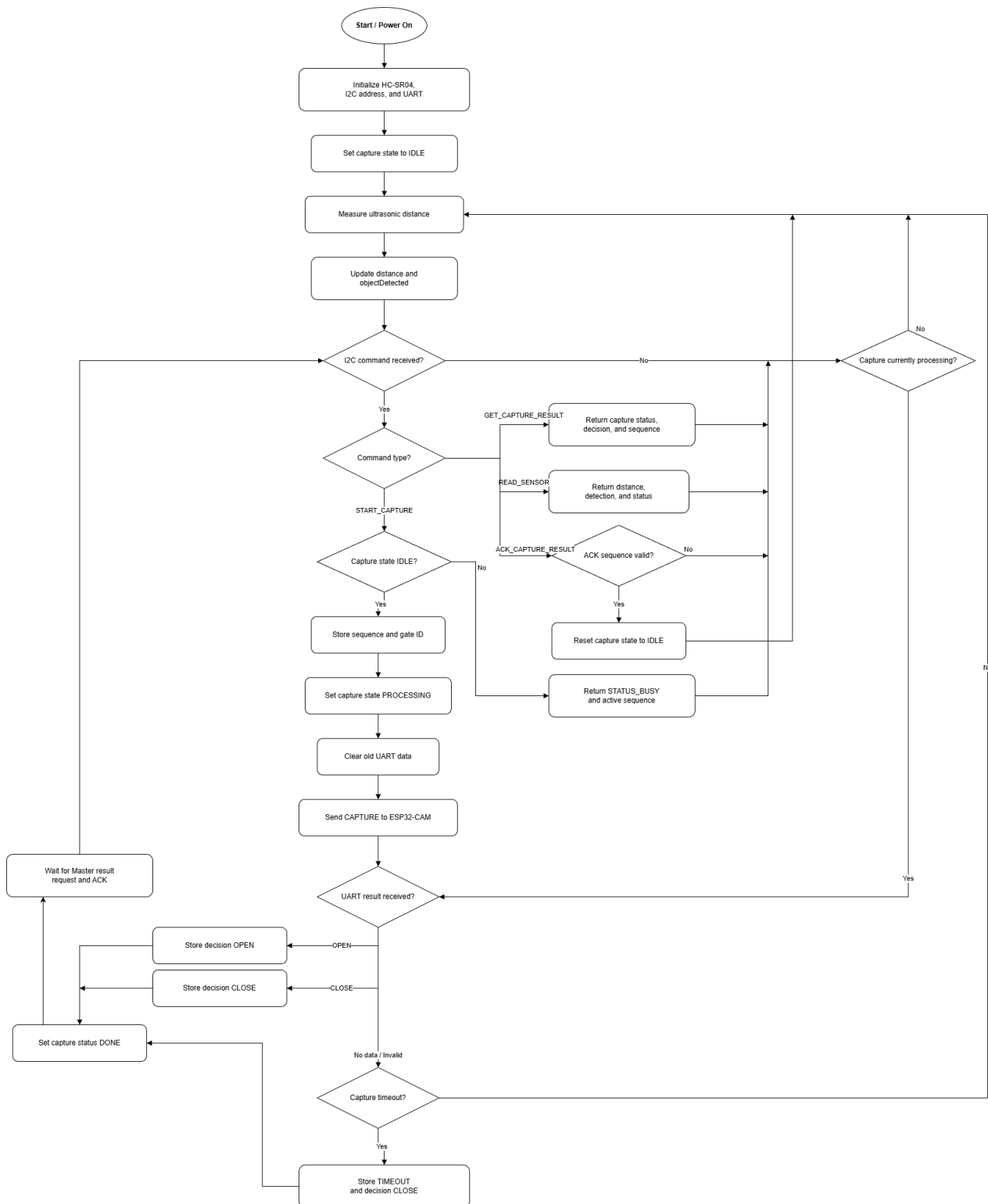


Fig. 4: Workflow of the Arduino Slave sensing and communication firmware.

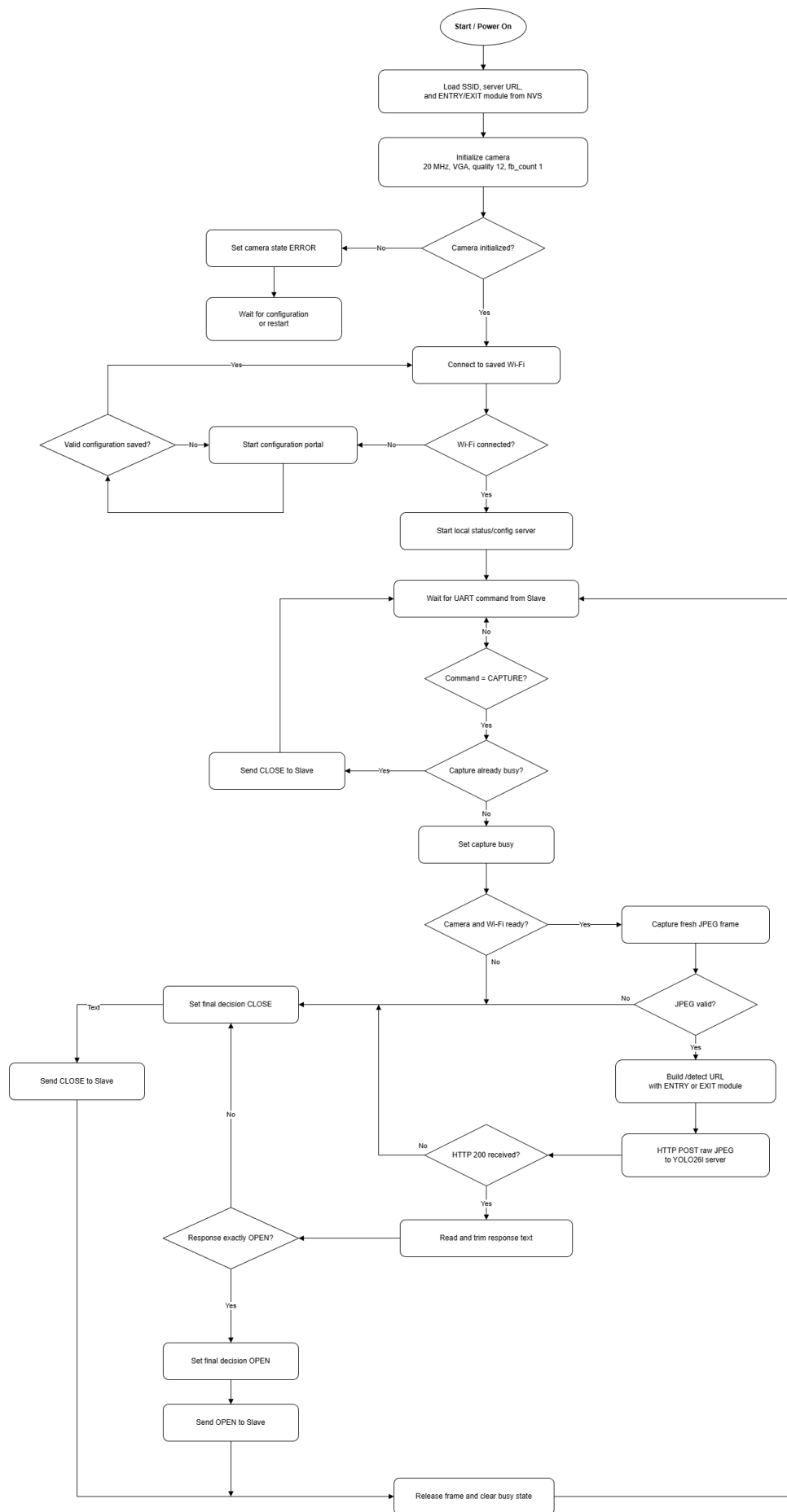


Fig. 5: Workflow of the ESP32-CAM image-capture and communication firmware.

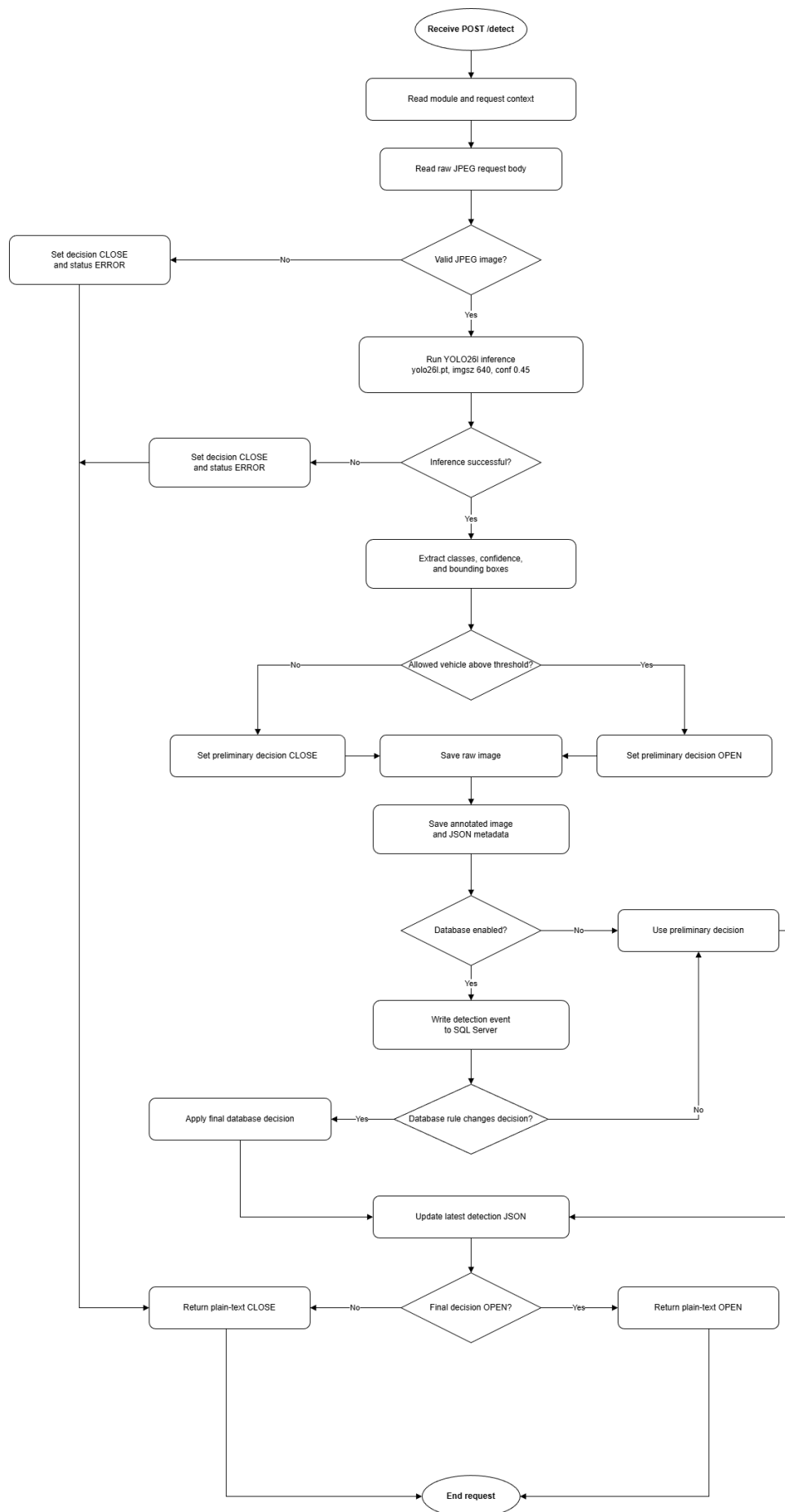


Fig. 6: Workflow of the FastAPI and YOLO26l vehicle-detection server.

ENTRY Slave uses I2C address 0×08 , while the EXIT Slave uses I2C address 0×09 . Each Slave is connected to one HC-SR04 ultrasonic sensor and one ESP32-CAM. The Master is connected to the SG90 servo barrier and the parallel LCD1602.

The camera modules transmit captured JPEG images through Wi-Fi to a local FastAPI backend. The backend runs the final YOLO26l vehicle-detection model stored in `yolo26l.pt`. When database logging is enabled, the backend stores available detection information in SQL Server. The Java dashboard retrieves the available records and displays the latest ENTRY and EXIT information.

Implementation was performed incrementally. Individual modules were initially tested separately, followed by communication testing between adjacent layers. The integration sequence was:

- 1) Arduino Master, servo, and LCD1602;
- 2) individual HC-SR04 and Arduino Slave modules;
- 3) Master-Slave I2C communication;
- 4) Slave-ESP32-CAM UART communication;
- 5) ESP32-CAM camera and Wi-Fi operation;
- 6) ESP32-CAM-FastAPI HTTP communication;
- 7) YOLO26l inference;
- 8) SQL Server logging;
- 9) Java dashboard retrieval;
- 10) complete embedded and server-side integration.

This incremental approach was necessary because the complete system combines different supply requirements, voltage levels, communication protocols, and timeout boundaries. A failure observed in the full prototype could originate from the sensor, I2C bus, UART interface, camera, Wi-Fi network, backend server, database, or power supply.

B. Hardware Prototype

1) *Physical Assembly*: The hardware was assembled as one integrated prototype rather than as multiple physically separated subsystem demonstrations. The assembly contains the Arduino Master, two Arduino Slaves, two HC-SR04 sensors, two ESP32-CAM modules, the LCD1602, the SG90 servo, and the shared power wiring.

Fig. 7 is used as the primary physical implementation evidence. Separate photographs of the Master, ENTRY module, EXIT module, LCD subsystem, or power-distribution subsystem are not required because these components are wired as one integrated assembly.

The prototype represents a small parking-gate environment using physical vehicle models. The SG90 servo represents the barrier actuator, while the HC-SR04 sensors represent the ENTRY and EXIT detection zones.

The ESP32-CAM units are positioned to capture images when the corresponding directional module begins a verification cycle.

2) *Central Master Assembly*: The central control assembly contains:

- one Arduino Uno R3 Master;
- one SG90 servo;
- one parallel LCD1602;
- the shared I2C SDA and SCL connections;
- the synchronization output;
- the common 5 V and ground connections.

The Arduino Master generates the PWM signal for the SG90 servo. No Slave or ESP32-CAM signal is connected directly to the servo-control input. This ensures that the Master remains the only module capable of changing the barrier position.

The LCD1602 was connected in 4-bit mode using the following implemented mapping:

TABLE XIII: Implemented LCD1602 Connection

Master Pin	LCD Pin	Implementation Role
D9	RS	Register selection
D10	E	Enable signal
D5	DB4	Parallel data bit
D6	DB5	Parallel data bit
D7	DB6	Parallel data bit
D8	DB7	Parallel data bit
GND	R/W	Fixed write mode
Potentiometer	VO	Contrast adjustment
5 V	VDD, LED+	LCD and backlight supply
GND	VSS	Common ground

The LCD displayed local diagnostic information during development, including measured distance, directional module state, capture state, returned decision, and barrier status. It was not implemented as the main user-facing system interface.

3) *ENTRY and EXIT Assemblies*: The ENTRY and EXIT assemblies use the same hardware structure but different identities.

Each Slave measures its local HC-SR04 sensor independently. When the Master starts a verification cycle, the active Slave sends a UART capture command to the corresponding ESP32-CAM. The camera identity and I2C address must refer to the same direction throughout the complete communication path.

The implementation did not include an IR receiver, IR remote, HC-SRF05 sensor, or external status LED. These components were removed from the final system scope.

4) *Power Implementation*: The final power design uses a regulated 5 V source with approximately 3 A ca-

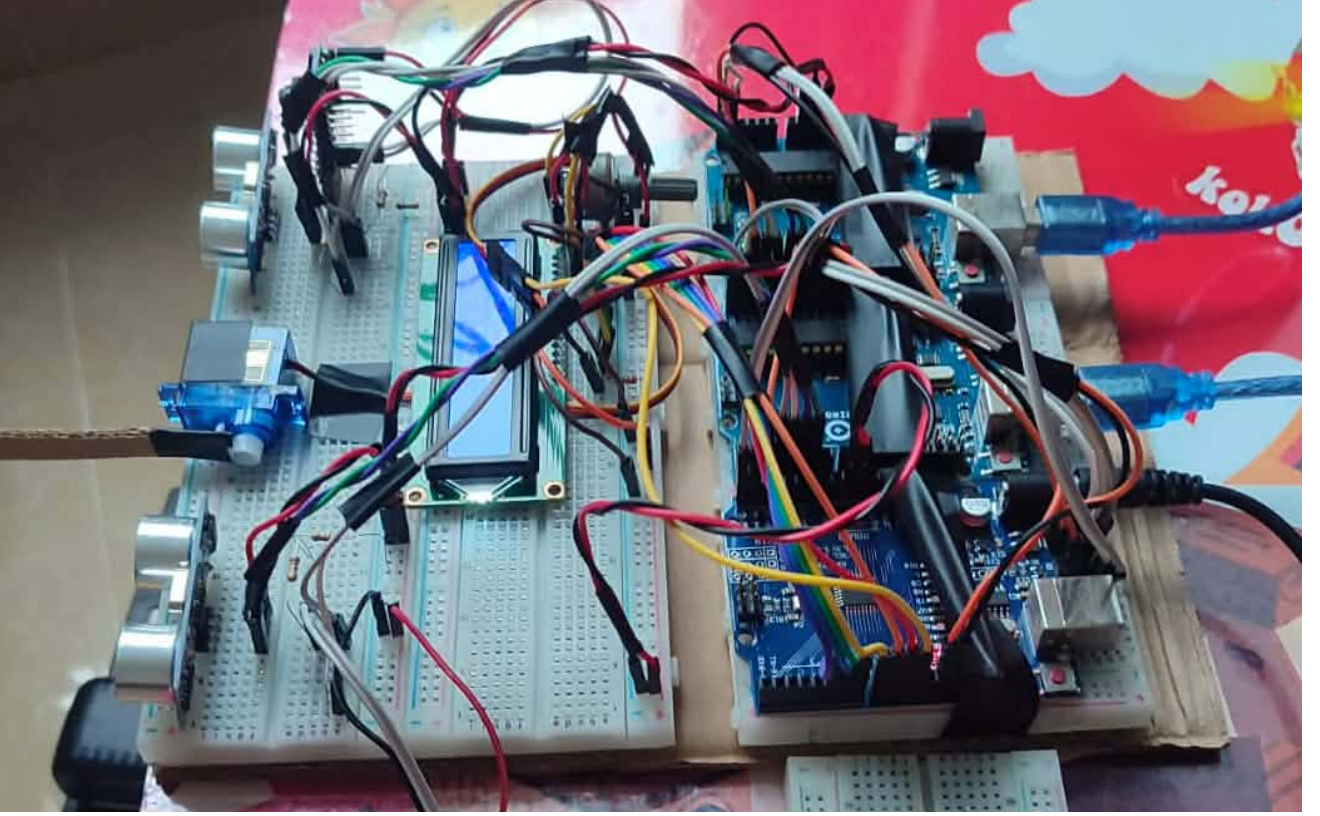


Fig. 7: Complete hardware prototype of the proposed smart parking system.

TABLE XIV: Implemented Directional Modules

Direction	Slave Address	Local Sensor	Camera Identity	Server Request
ENTRY	0x08	HC-SR04 ENTRY sensor	ENTRY	/detect?module=ENTRY
EXIT	0x09	HC-SR04 EXIT sensor	EXIT	/detect?module=EXIT

capacity. All Arduino boards, sensors, ESP32-CAM modules, the servo, and the LCD share a common ground.

The ESP32-CAM and servo require particular attention because both can create short changes in current demand. Wi-Fi initialization, camera capture, JPEG buffering, and servo movement may cause voltage reduction when the supply path has excessive resistance or insufficient transient-current capacity.

The final implementation therefore requires:

- a regulated 5 V supply;
- short power branches;
- sufficiently sized conductors;
- common ground between all modules;
- stable power at the ESP32-CAM 5 V input;
- stable power at the servo supply;
- separation of high-current servo wiring from sensitive signal wiring where possible.

The previous 12 V/2 A adapter path was not used as the final supply configuration. Although its nominal wattage was sufficient, the associated conversion and

wiring path did not provide a stable 5 V rail for the ESP32-CAM under the implemented conditions. The ESP32-CAM operated correctly through laptop USB power but was unstable through the previous adapter path. The implementation issue was therefore associated with voltage conversion, voltage drop, and transient-current delivery rather than the nominal adapter power rating alone.

5) *UART Voltage-Level Protection:* The Arduino Uno uses 5 V logic, while the ESP32-CAM uses 3.3 V logic. Directly connecting the Arduino Slave TX output to the ESP32-CAM RX input can expose the ESP32 input to a voltage above its intended logic level.

The implemented electrical design therefore requires a resistor voltage divider or equivalent level-shifting circuit on:

```

Arduino Slave TX
-> voltage divider
-> ESP32-CAM RX

```


The ESP32-CAM TX output uses approximately 3.3 V logic and can be read by the Arduino Uno RX input. All UART devices must share a common ground.

C. Arduino Master Implementation

1) *Firmware Structure*: The Master firmware was implemented in `Master.ino`. It performs system initialization, I2C polling, capture-cycle management, result validation, servo control, object-clear handling, and LCD updates.

The principal initialization sequence is conceptually represented by:

```
initialize I2C
initialize LCD1602
attach SG90 servo
set barrier CLOSED
reset ENTRY state
reset EXIT state
reset active capture state
start periodic polling
```

The servo is moved to the closed position during startup. This establishes a deterministic initial state and prevents the barrier from remaining in an undefined position after power-on or reset.

2) *Polling Implementation*: The Master polls the ENTRY and EXIT Slaves every 1000 ms. The polling logic requests the current directional module state and stores the returned information only when a valid response is available.

The Master distinguishes the two modules using:

```
ENTRY_ADDRESS = 0x08
EXIT_ADDRESS  = 0x09
```

The module state contains or represents the current distance, object-detection condition, capture status, and result. The exact byte representation is handled by the I2C protocol implemented in the Master and Slave firmware.

The 1000 ms interval is a scheduling configuration and was not used as a measured communication latency. The effective waiting time before the Master observes a new detection depends on where the event occurs within the polling cycle.

3) *Capture Request Implementation*: When the Master identifies a valid object event, it selects the active direction and sends a capture request to the corresponding Slave. Before sending the request, the Master clears the previously stored result for that direction.

This reset is important because a previous OPEN value must not be interpreted as the result of the new event. The Master then changes to a result-waiting state.

The configured waiting limit is approximately 7500–8000 ms. If a valid result is not available before this timeout expires, the Master ends the cycle with a closed decision.

4) *Barrier Implementation*: The SG90 servo is controlled exclusively by the Master. The barrier-control implementation follows these conditions:

- 1) The barrier begins closed.
- 2) A detection flag alone does not open the barrier.
- 3) A valid current OPEN result allows the barrier to open.
- 4) A CLOSE result keeps the barrier closed.
- 5) A timeout keeps the barrier closed.
- 6) Invalid I2C data keeps the barrier closed.
- 7) A result from a previous cycle must not open the barrier.

The minimum barrier-open interval is approximately 3000 ms. This configuration prevents the barrier from closing immediately after an open command. However, a fixed timer alone is insufficient for full-size vehicle safety. The final control design should also verify that the object has cleared the detection region before closing, subject to a maximum-open safety timeout.

5) *Master Diagnostic Output*: The Master generates diagnostic output through the LCD and serial interface during testing. Typical information includes:

- ENTRY distance;
- EXIT distance;
- ENTRY detection state;
- EXIT detection state;
- Slave status;
- active direction;
- capture state;
- OPEN or CLOSE result;
- barrier state.

An invalid or unavailable distance may be represented by a value such as 999 cm. This value is used as a diagnostic indication and is not interpreted as a nearby object.

D. Arduino Slave Implementation

1) *Shared Slave Firmware*: The directional firmware was implemented in `Slave.ino`. The same general firmware structure is used for both directions, while the I2C address and module configuration distinguish ENTRY from EXIT.

Each Slave performs four principal functions:

- 1) HC-SR04 distance measurement;
- 2) object-state calculation;
- 3) I2C communication with the Master;
- 4) UART communication with the ESP32-CAM.

The Slave continuously updates its sensor state independently of the Master polling interval. When the Master requests data, the most recent available state is returned.

2) *HC-SR04 Implementation*: The Slave generates a trigger pulse and measures the echo duration using the configured trigger and echo pins. The echo duration is converted into a distance value.

The sensor implementation distinguishes between:

- valid distance within the detection region;
- valid distance outside the detection region;
- missing or timed-out echo;
- invalid or unavailable measurement.

A missing echo does not produce a positive object-detection result. The detection threshold is a firmware parameter and is not reported as measured sensor accuracy.

3) *I2C Request and Receive Handlers*: The Slave firmware registers one handler for receiving commands from the Master and one handler for responding to Master requests.

The receive handler processes control information such as:

- start capture;
- reset or acknowledgement;
- synchronization-related control where implemented.

The request handler returns the current local state. The Slave must avoid performing long blocking camera or UART operations directly inside the I2C interrupt callback. Instead, the callback updates flags, while the main loop performs the longer processing stages.

4) *UART Capture Implementation*: When a capture flag is set, the Slave sends:

CAPTURE

to the corresponding ESP32-CAM at 9600 baud.
The Slave then waits for:

OPEN

or:

CLOSE

Incoming UART data is accumulated into a bounded text buffer. Carriage-return and newline characters are used to identify message completion. The message is trimmed before comparison.

Unknown text is not considered a valid result. This is important because serial debug output may otherwise be mistaken for a decision. If no valid result is received within approximately 6000 ms, the Slave stores a closed result.

5) *Result Latching*: After receiving a valid result, the Slave exposes it to the Master through I2C. The result should remain available until the Master confirms that it has processed the active cycle.

Prematurely clearing the result after the first I2C read could cause a lost decision if the read is incomplete. Therefore, the implementation should retain the decision until the corresponding acknowledgement or cycle reset is processed.

The final report does not claim that every acknowledgement edge case was quantitatively validated. Result acknowledgement remains an area requiring additional verification.

E. ESP32-CAM Implementation

1) *Firmware Structure*: The ESP32-CAM implementation handles camera initialization, configuration persistence, Wi-Fi connection, UART command processing, JPEG capture, HTTP communication, response normalization, and fail-close behavior.

The firmware stores or uses the following configuration values:

- Wi-Fi SSID;
- Wi-Fi password;
- backend server URL;
- module identity;
- configuration-valid state.

The module identity is configured as either ENTRY or EXIT. This identity determines the HTTP query parameter and the direction recorded by the backend.

2) *Final Camera Configuration*: The implemented final camera settings are:

```
config.xclk_freq_hz = 20000000;
config.frame_size = FRAMESIZE_VGA;
config.jpeg_quality = 12;
config.fb_count = 1;
```

The external camera clock is configured at 20 MHz. VGA resolution is used to provide a low-resolution image suitable for transmission to the local backend. JPEG quality value 12 and one frame buffer are used in the final configuration.

These settings are implementation parameters. They do not represent measured image quality, capture speed, or system latency.

3) *Wi-Fi and Configuration Startup*: During startup, the ESP32-CAM loads the stored configuration and attempts to connect to the configured Wi-Fi network. A module-specific hostname may be used to distinguish the cameras on the local network.

The configuration remains in non-volatile storage until it is explicitly changed or cleared. Reflashing the firmware may not automatically erase the stored Wi-Fi and server configuration.

The current final scope does not include periodic heartbeat transmission. The implementation must therefore not be described as sending regular device-health records to the backend.

4) *UART Command Processing*: The ESP32-CAM waits for a normalized CAPTURE command from the Arduino Slave. Unrecognized UART input does not start image capture.

When a valid command is received, the ESP32-CAM verifies:

- 1) the camera is initialized;
- 2) the module identity is valid;
- 3) the backend URL is valid;
- 4) Wi-Fi is connected.

If any condition is false, the module returns CLOSE without producing an open result.

5) *JPEG Capture*: The camera driver returns a frame-buffer pointer containing the captured JPEG data. The implementation checks whether:

- the frame-buffer pointer is valid;
- the image length is greater than zero;
- the image data is available;
- the frame uses the expected JPEG representation.

When capture fails, the ESP32-CAM releases the frame buffer where required and sends CLOSE through UART.

6) *HTTP Upload*: The ESP32-CAM submits the JPEG to one of the following routes:

```
/detect?module=ENTRY
```

or:

```
/detect?module=EXIT
```

The HTTP timeout is configured at approximately 3500 ms. The camera accepts a successful transaction only when the server returns HTTP status 200 and a valid decision body.

The decision normalization is intentionally conservative:

```
if response == "OPEN":
    send "OPEN"
else:
    send "CLOSE"
```

Therefore, a network error, empty response, HTML error page, malformed string, unknown decision, or explicit CLOSE result keeps the barrier closed.

F. FastAPI and YOLO26l Implementation

1) *Backend Environment*: The backend was implemented in Python using FastAPI and the Ultralytics YOLO library. The server runs on a local computer connected to the same network as the ESP32-CAM modules.

The final model is loaded using:

```
from ultralytics import YOLO

model = YOLO("yolo26l.pt")
```

The backend exposes the detection endpoint and processes JPEG request bodies received from the camera modules.

2) *Request Validation*: The detection endpoint validates the following inputs:

- module query parameter;
- request-body length;
- JPEG decoding result;
- supported direction;
- availability of the loaded YOLO model.

The module value must be either ENTRY or EXIT. Invalid directions are rejected or converted to a closed decision.

An empty or invalid image does not proceed to a valid open result.

3) *YOLO26l Inference*: The final inference configuration is:

TABLE XV: Implemented YOLO26l Parameters

Parameter	Value
Model	yolo26l.pt
Inference image size	640 pixels
Confidence threshold	0.45
Allowed class	car
Allowed class	motorcycle
Allowed class	bus
Allowed class	truck
Positive response	OPEN
Negative response	CLOSE

The server iterates through the YOLO detection results and checks whether at least one allowed class satisfies the configured confidence threshold.

The decision condition is:

$$D = \begin{cases} \text{OPEN}, & \exists b : C_b \in C_{\text{allowed}} \wedge p_b \geq 0.45, \\ \text{CLOSE}, & \text{otherwise,} \end{cases} \quad (5)$$

where b is a detection bounding box, C_b is its predicted class, p_b is its confidence score, and C_{allowed} is the set containing car, motorcycle, bus, and truck.

The detection-confidence score is stored as model output evidence. It is not interpreted as complete model accuracy.

4) *Image Output*: The server creates direction-specific file names for raw and annotated images. Relative paths are used so that the records can be consumed by the dashboard without storing a workstation-specific absolute path.

The server output structure may include directories such as:

```
parking-image/raw/
parking-image/results/annotated/
```

The annotated image contains the model bounding box, detected class, and confidence value where a valid detection is available.



Fig. 8: Example annotated vehicle-detection result produced by YOLO26l.

Fig. 8 provides visual implementation evidence for the server-side detection process. It should contain an actual result generated from the ESP32-CAM workflow.

5) *FastAPI Log Evidence*: The server terminal records information such as the module direction, request processing, model selection, inference time, detected class, confidence value, output path, and final decision.

The log shown in Fig. 9 should correspond to a real processed request. Historical YOLOv8n timing must not be presented as YOLO26l output.

6) *Backend Failure Handling*: The implemented backend returns or produces a closed result when:

- the module identity is invalid;
- the image body is empty;
- the JPEG cannot be decoded;
- model inference raises an exception;
- no allowed vehicle class is detected;

```
2026-07-15 21:48:38.833 | INFO | Loading YOLO model: yolo26l.pt
2026-07-15 21:48:39.295 | INFO | YOLO model loaded. Class names: {0: 'person', 1: 'bicycle', 2: 'car',
3: 'motorcycle', 4: 'airplane', 5: 'bus', 6: 'train', 7: 'truck', 8: 'boat', 9: 'traffic light', 10:
'fire hydrant', 11: 'stop sign', 12: 'parking meter', 13: 'bench', 14: 'bird', 15: 'cat', 16: 'dog', 17:
horse', 18: 'sheep', 19: 'cow', 20: 'elephant', 21: 'bear', 22: 'zebra', 23: 'giraffe', 24: 'backpack',
25: 'umbrella', 26: 'handbag', 27: 'tie', 28: 'suitcase', 29: 'frisbee', 30: 'skis', 31: 'snowboard',
32: 'sports ball', 33: 'kite', 34: 'baseball bat', 35: 'baseball glove', 36: 'skateboard', 37: 'surfboard',
38: 'tennis racket', 39: 'bottle', 40: 'wine glass', 41: 'cup', 42: 'fork', 43: 'knife', 44: 'spoon', 45:
'bow', 46: 'banana', 47: 'apple', 48: 'sandwich', 49: 'orange', 50: 'broccoli', 51: 'carrot', 52: 'hot dog',
53: 'pizza', 54: 'donut', 55: 'cake', 56: 'chair', 57: 'couch', 58: 'potted plant', 59: 'bed', 60: 'dining table',
61: 'toilet', 62: 'tv', 63: 'laptop', 64: 'mouse', 65: 'remote', 66: 'keyboard', 67: 'cell phone', 68: 'microwave',
69: 'oven', 70: 'toaster', 71: 'sink', 72: 'refrigerator', 73: 'book', 74: 'clock', 75: 'vase', 76: 'scissors', 77: 'teddy bear',
78: 'hair drier', 79: 'toothbrush'}
2026-07-15 21:48:39.295 | INFO | Allowed classes for OPEN: ['bus', 'car', 'motorcycle', 'truck']
2026-07-15 21:48:39.296 | INFO | Database logging enabled: False
INFO: Started server process [27348]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8080 (Press CTRL+C to quit)
2026-07-15 21:49:37.415 | INFO | request_id=20260715_214936_537596 direction=ENTRY seq=None decision=0
PEN best={'class_id': 2, 'class': 'car', 'confidence': 0.9601, 'confidence_percent': 96.01, 'bbox': [1
94.19, 241.57, 335.48, 376.07]}, 'allowed': True} db_enabled=False inference_ms=827.14
INFO: 127.0.0.1:621773 - "POST /detect?module=ENTRY HTTP/1.1" 200 OK
2026-07-15 21:49:38.911 | INFO | request_id=20260715_214938_393121 direction=ENTRY seq=None decision=0
PEN best={'class_id': 2, 'class': 'car', 'confidence': 0.9655, 'confidence_percent': 96.55, 'bbox': [1
44.48, 130.21, 636.84, 384.28]}, 'allowed': True} db_enabled=False inference_ms=582.1
INFO: 127.0.0.1:62187 - "POST /detect?module=ENTRY HTTP/1.1" 200 OK
2026-07-15 21:49:40.282 | INFO | request_id=20260715_214939_880582 direction=ENTRY seq=None decision=0
PEN best={'class_id': 2, 'class': 'car', 'confidence': 0.969, 'confidence_percent': 96.9, 'bbox': [60.
23, 71.72, 689.36, 326.65]}, 'allowed': True} db_enabled=False inference_ms=386.95
INFO: 127.0.0.1:61308 - "POST /detect?module=ENTRY HTTP/1.1" 200 OK
2026-07-15 21:49:41.755 | INFO | request_id=20260715_214941_264448 direction=ENTRY seq=None decision=0
PEN best={'class_id': 2, 'class': 'car', 'confidence': 0.9462, 'confidence_percent': 94.62, 'bbox': [2
0.25, 67.45, 471.54, 334.59]}, 'allowed': True} db_enabled=False inference_ms=478.57
INFO: 127.0.0.1:61313 - "POST /detect?module=ENTRY HTTP/1.1" 200 OK
2026-07-15 21:49:43.412 | INFO | request_id=20260715_214942_871801 direction=ENTRY seq=None decision=0
PEN best={'class_id': 7, 'class': 'truck', 'confidence': 0.8557, 'confidence_percent': 85.57, 'bbox': [
275.47, 332.32, 797.04, 599.38]}, 'allowed': True} db_enabled=False inference_ms=521.3
INFO: 127.0.0.1:61320 - "POST /detect?module=ENTRY HTTP/1.1" 200 OK
```

Fig. 9: FastAPI server output during YOLO26l vehicle-detection processing.

- every allowed detection is below the threshold;
- an unexpected processing error occurs.

The backend does not send a direct command to the servo. Its result travels through the ESP32-CAM and Arduino Slave before the Master makes the actuator decision.

G. SQL Server Implementation

1) *Database Structure*: The database was implemented using the SmartParkingIOT102 SQL Server database. The core schema contains:

- parking_occupancy;
- gates;
- detection_events.

The parking_occupancy table stores one current occupancy record containing the configured capacity, number of vehicles inside, and update timestamp.

The gates table stores the logical state of the ENTRY and EXIT gates. The identity mapping is:

```
gate_id = 1 -> ENTRY
gate_id = 2 -> EXIT
```

The detection_events table stores available server-side event evidence.

2) *Database Constraints*: The occupancy table uses constraints to prevent:

- negative parking capacity;
- negative vehicle count;
- vehicle count greater than capacity.

The detection-events table references the gate table so that each stored event is associated with a valid direction.

TABLE XVI: Implemented Detection Event Fields

Field	Data Role	Stored Information
event_id	Primary key	Unique identifier for the detection event.
gate_id	Foreign key	Directional gate identifier linked to the <code>gates</code> table.
created_at	Timestamp	Server-side event creation time.
decision	Decision	Final OPEN or CLOSE result.
confidence	Model output	Confidence value of the accepted vehicle detection, where available.
class_name	Model output	Accepted detected vehicle class, where available.
raw_image_path	Relative path	Location of the original image received from the ESP32-CAM.
annotated_image_path	Relative path	Location of the annotated server output.

The database records server-side events. They do not independently confirm the physical movement of the SG90 barrier because complete Master gate-state telemetry is not transmitted to the backend in the current implementation.

Fig. 10: SQL Server detection record generated by the implemented backend.

3) *SQL Record Evidence*: Fig. 10 should show an actual `detection_events` record, including the direction, decision, class, confidence, timestamp, and image path where those values are available.

The database implementation was not evaluated through sufficient repeated record-validation trials to claim complete logging reliability or occupancy accuracy.

H. Java Web Dashboard Implementation

1) *Application Structure*: The dashboard was implemented as a Java Servlet and JSP application. The main request path is:

/dashboard

The principal controller is:

DashboardController

The controller coordinates the following DAO classes:

- DashboardDAO;
- DetectionEventDAO;
- GateDAO;
- ParkingOccupancyDAO.

The returned database values are mapped to:

- DashboardOverview;

- DetectionEvent;
- Gate;
- ParkingOccupancy.

The controller places the objects in request attributes and forwards the request to `dashboard.jsp`.

2) *Dashboard Content*: The dashboard was designed to display available information including:

- total capacity;
- vehicles inside;
- available capacity;
- current gate state;
- latest ENTRY event;
- latest EXIT event;
- event decision;
- detected class;
- confidence value;
- event timestamp;
- raw or annotated image.

Fig. 11 should contain the actual dashboard output. The dashboard is used for monitoring and does not directly control the SG90 servo.

3) *Image-Serving Servlet*: The images generated by the YOLO server are located outside the Java web application's static directory. Directly embedding a local Windows file-system path in JSP would not allow the browser to retrieve the image.

An image-serving servlet is therefore used to:

- 1) receive a relative image path;
- 2) normalize the requested path;
- 3) reject invalid or traversal paths;
- 4) resolve the path under the configured image base directory;
- 5) verify that the image exists;
- 6) set the response MIME type;
- 7) stream the image bytes to the browser.

The JSP references the servlet URL rather than an absolute development-machine path. This separates the database path representation from the browser-accessible resource URL.

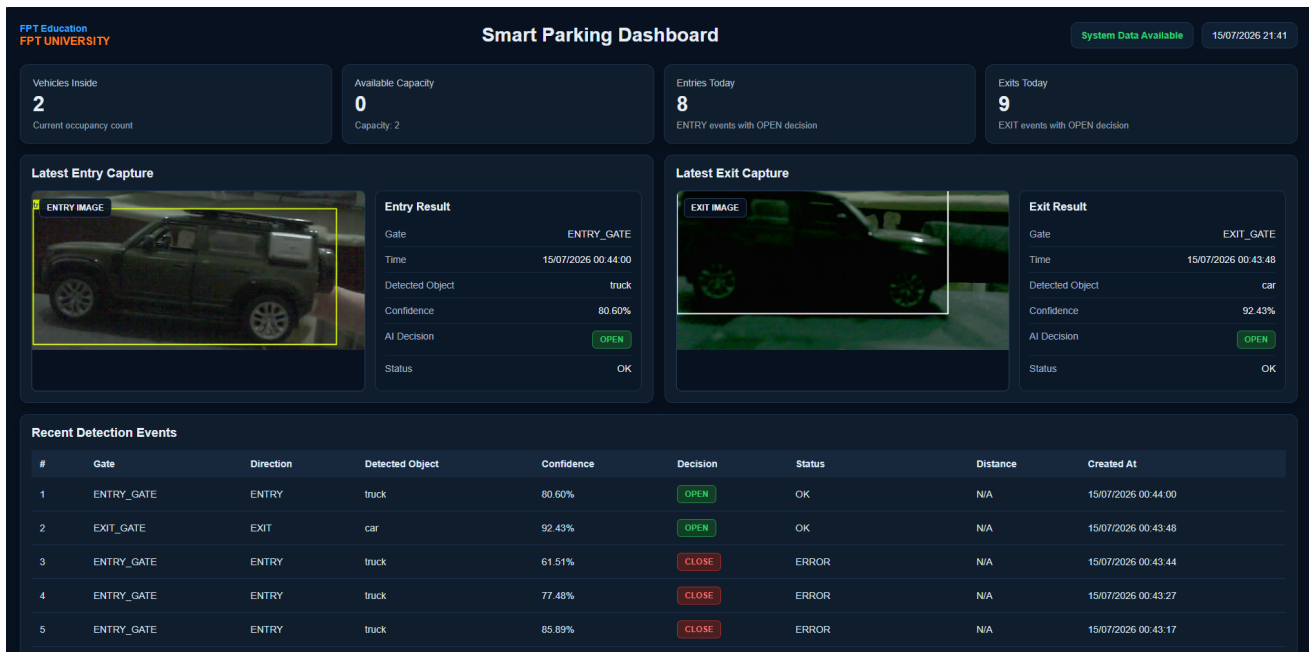


Fig. 11: Overview page of the implemented smart parking dashboard.

4) *Dashboard Evidence Limitation:* The dashboard and database implementation provides evidence of integration between the backend and monitoring application. However, the available evidence does not support claims of:

- measured dashboard-update delay;
- complete long-duration dashboard availability;
- perfect event-record completeness;
- verified occupancy accuracy;
- production access control;
- full synchronization with the physical gate state.

The dashboard should therefore be described as a demonstrated monitoring component rather than a fully validated production monitoring platform.

I. Integrated Communication Implementation

1) *Master-Slave Integration:* The Master and both Slaves share the I2C SDA and SCL lines and use different addresses. The synchronization connection is implemented from Master D3 to Slave D2 according to the current hardware design.

The normal communication sequence is:

```
Master polls 0x08
-> ENTRY Slave response
```

```
Master polls 0x09
-> EXIT Slave response
```

When both Slaves were active and responsive, communication succeeded during the observed normal-condition

tests. When either Slave was inactive, the current Master implementation stopped normal operation. This behavior is reported as a module-dependency limitation rather than fault-tolerant operation.

2) *Slave-Camera Integration:* Each Slave communicates only with its associated ESP32-CAM. The direction mapping must not be crossed during wiring or configuration.

The implemented protocol uses:

```
Slave -> ESP32-CAM: CAPTURE
ESP32-CAM -> Slave: OPEN
ESP32-CAM -> Slave: CLOSE
```

The protocol is intentionally simple, but it provides limited event context. It does not carry a full sequence identifier, measured distance, timestamp, or sensor state. The Master and Slave state machines must therefore prevent a stale decision from being associated with a later event.

3) *Camera-Server Integration:* The ESP32-CAM communicates with the backend through HTTP over the local Wi-Fi network. Successful operation requires:

- both devices on reachable network segments;
- a valid backend IP address;
- the correct server port;
- an available FastAPI process;
- host-firewall permission;
- valid module query parameters;
- stable ESP32-CAM power.

An HTTP client result such as code `-1` indicates that the request did not receive a successful HTTP response. Possible causes include an unreachable server, incorrect URL, blocked port, Wi-Fi disconnection, or network failure. Such a condition produces a closed decision.

J. Implementation Challenges and Corrective Actions

The prototype required integration across hardware, firmware, networking, AI processing, database access, and web presentation. Table XVII summarizes the main implementation challenges and the corrective actions applied or identified.

K. Implementation Verification Checklist

Before an integrated demonstration, the implementation should be checked using the conditions in Table XVIII.

L. Implementation Summary

The implementation produced an integrated smart parking prototype that connects ultrasonic sensing, Arduino Master-Slave coordination, ESP32-CAM image acquisition, FastAPI and YOLO26l processing, SQL Server persistence, dashboard retrieval, and servo-barrier control.

The system maintains explicit directional identities for ENTRY and EXIT and uses a layered communication structure consisting of I2C, UART, and HTTP. The Arduino Master retains exclusive control of the SG90 servo, while all uncertain conditions follow a fail-close rule.

The implementation also exposed several practical constraints. Stable ESP32-CAM power, UART voltage compatibility, correct module mapping, coordinated timeouts, network reachability, and image quality were all necessary for successful integration. The current prototype demonstrates the implemented control concept, but additional fault isolation, synchronized timing instrumentation, dashboard validation, negative-sample testing, and full-size safety evaluation are required before real-world deployment.

VI. RESULTS AND DISCUSSION

A. Evaluation Scope

The evaluation focused on the evidence that could be reproduced from the implemented prototype and retained experimental records. The available results were divided into three categories:

- 1) Server-side YOLO26l vehicle-detection results obtained from stored JSON logs;

- 2) Historical and same-source comparison results between YOLOv8n and YOLO26l;
- 3) Observed Arduino Master-Slave communication behavior during integration testing.

The evaluation did not treat configuration parameters as measured performance. Values such as the 1000 ms Master polling interval, 3500 ms ESP32-CAM HTTP timeout, 6000 ms Slave capture timeout, 7500–8000 ms Master capture timeout, and 3000 ms minimum barrier-open time are software settings rather than experimental response-time measurements.

The main YOLO26l experiment used 146 real images captured through the ESP32-CAM workflow. The images contained physical vehicle models and were not synthetically generated. Every image in the dataset was described as containing a vehicle model. Therefore, this dataset is a positive-only dataset.

The positive-only structure allows the following metrics to be reported:

- positive-sample vehicle-detection rate;
- miss rate on confirmed positive samples;
- number of OPEN and CLOSE decisions;
- accepted predicted vehicle classes;
- confidence statistics for accepted detections;
- server-side inference-time statistics.

The dataset does not support complete classification accuracy, precision, specificity, false-positive rate, or F1-score because it contains no confirmed negative samples and no complete manually verified confusion matrix.

B. Experimental Configuration

The final server-side vehicle-detection model was YOLO26l, loaded from the model file `yolo26l.pt`. The inference configuration is summarized in Table XIX.

The server returned OPEN when at least one allowed vehicle class had a confidence score greater than or equal to 0.45. If no allowed detection satisfied this rule, the server returned CLOSE.

The decision rule can be expressed as:

$$D = \begin{cases} \text{OPEN}, & \exists i : c_i \in C_{\text{vehicle}} \wedge p_i \geq 0.45, \\ \text{CLOSE}, & \text{otherwise,} \end{cases} \quad (6)$$

where D is the server decision, c_i is the predicted class of detection i , p_i is its confidence score, and C_{vehicle} contains car, motorcycle, bus, and truck.

C. Overall YOLO26l Results

The final YOLO26l experiment contained 146 result logs. The server produced 66 OPEN decisions and 80 CLOSE decisions.

TABLE XVII: Implementation Challenges and Corrective Actions

Challenge	Observed Condition	Implementation Impact	Corrective Action
Pin and direction consistency	Earlier design artifacts contained inconsistent ENTRY and EXIT mapping.	Incorrect requests or responses could be associated with the wrong physical direction.	Standardize ENTRY as 0x08 and EXIT as 0x09 across firmware, circuit, camera configuration, server, database, and dashboard.
ESP32-CAM power stability	The camera operated from laptop USB but was unstable through the previous adapter and conversion path.	Camera initialization, Wi-Fi connection, or HTTP upload could fail.	Use a regulated 5 V supply with sufficient transient-current capacity, short wiring, low voltage drop, and common ground.
Servo supply disturbance	Servo movement can create rapid current changes on the shared supply.	Voltage variation may affect the Arduino or ESP32-CAM.	Use a stable 5 V branch, short servo wiring, supply decoupling, and appropriate grounding.
UART voltage mismatch	Arduino TX uses 5 V logic, while the ESP32-CAM RX input uses 3.3 V logic.	Direct connection can cause unreliable operation or electrical damage.	Add a resistor divider or level shifter on Slave TX to ESP32-CAM RX.
Server reachability	Wi-Fi or server-address errors can produce an HTTP connection failure.	No vehicle-verification response reaches the embedded system.	Validate the complete server URL, network address, port, firewall, and FastAPI process; convert failure to CLOSE.
Camera image quality	Low-resolution vehicle-model images vary with angle, lighting, distance, and object scale.	YOLO26l may return CLOSE even when a vehicle model is present.	Use the final VGA configuration, stabilize camera placement and lighting, and document the observed miss rate.
YOLO processing cost	YOLO26l provides more usable detections in the reused test collection but requires greater inference time than YOLOv8n.	Backend processing increases the vehicle-verification delay.	Retain server-side inference, report the measured timing trade-off, and avoid treating inference time as total gate-response time.
Layered timeout coordination	Similar timeout values can create a race where an upper layer stops waiting before a lower layer stores the result.	A valid result may be converted into a timeout.	Use approximately 3500 ms for ESP HTTP, 6000 ms for Slave capture, and 7500–8000 ms for Master capture.
Stale result handling	A previous OPEN result could be unsafe if reused during a later cycle.	The barrier could open for the wrong event.	Clear previous results at cycle start and retain new results until the active cycle acknowledges them.
Repeated detection	The same object may remain inside the ultrasonic region.	The system could repeatedly capture images or reopen the barrier.	Add WAIT_OBJECT_CLEAR logic and allow a new cycle only after clearance.
Fixed barrier timer	A minimum hold time alone does not confirm that an object has passed.	The barrier may close before the object clears.	Combine minimum-open timing with object-clear confirmation and a maximum-open safety timeout.
Multi-Slave dependency	The Master stops normal operation when either Slave is inactive.	One failed direction prevents operation of the healthy direction.	Document the limitation and implement independent module-health and degraded-operation logic in future work.
Incomplete backend telemetry	The backend receives detection data but not complete Master servo and sensor state.	Dashboard records do not represent every physical state transition.	Add a future telemetry path from the embedded layer to the backend.
Limited dashboard validation	Available screenshots and repeated record-validation evidence are incomplete.	Complete monitoring reliability cannot be established.	Report only demonstrated functions and conduct repeated database and dashboard validation in future tests.

TABLE XVIII: Integrated Implementation Verification Checklist

ID	Verification Item	Required Condition
I01	ENTRY Slave identity	Address 0x08; camera module ENTRY.
I02	EXIT Slave identity	Address 0x09; camera module EXIT.
I03	I2C bus	Shared SDA/SCL and common ground; both Slaves responsive.
I04	Synchronization wiring	Master D3 connected consistently to Slave D2 according to the implemented circuit.
I05	UART voltage level	Level reduction installed on Arduino TX to ESP32-CAM RX.
I06	ESP32-CAM configuration	20 MHz XCLK, VGA, JPEG quality 12, one frame buffer.
I07	Wi-Fi configuration	Correct SSID, password, IP network, and module hostname.
I08	Server URL	Complete valid backend address without duplicated port or malformed endpoint.
I09	Backend model	yolo261.pt loaded successfully.
I10	Detection parameters	Image size 640, confidence threshold 0.45, allowed vehicle classes configured.
I11	FastAPI availability	Detection endpoint reachable from both ESP32-CAM modules.
I12	SQL Server configuration	Connection available when database logging is enabled.
I13	Dashboard connection	Java application can retrieve available SQL Server records.
I14	Image serving	Relative raw and annotated paths can be streamed through the web application.
I15	Barrier startup	Servo begins in the closed position.
I16	Fail-close handling	Camera, Wi-Fi, HTTP, server, UART, and timeout failures do not open the barrier.
I17	Object-clear behavior	A completed event cannot continuously restart while the object remains detected.

TABLE XIX: YOLO26l Experimental Configuration

Parameter	Configured Value
Model	YOLO26l
Model file	yolo261.pt
Input image source	ESP32-CAM workflow
Input image type	Real images of physical vehicle models
Inference image size	640 pixels
Confidence threshold	0.45
Allowed classes	Car, motorcycle, bus, truck
Positive decision	OPEN
Negative decision	CLOSE

The positive-sample vehicle-detection rate was calculated using:

$$R_{\text{detect}} = \frac{N_{\text{OPEN}}}{N_{\text{positive}}} \times 100\%, \quad (7)$$

where N_{OPEN} is the number of positive images that produced an accepted vehicle detection, and N_{positive} is the total number of confirmed positive input images.

For the final dataset:

$$R_{\text{detect}} = \frac{66}{146} \times 100\% = 45.21\%. \quad (8)$$

The miss rate was calculated using:

$$R_{\text{miss}} = \frac{N_{\text{CLOSE}}}{N_{\text{positive}}} \times 100\%. \quad (9)$$

For the final dataset:

$$R_{\text{miss}} = \frac{80}{146} \times 100\% = 54.79\%. \quad (10)$$

Table XX presents the complete verified result summary.

The 45.21% result is a positive-sample detection rate and must not be described as overall model accuracy. An overall accuracy calculation would require both positive and negative samples with manually confirmed ground-truth labels.

The 54.79% miss rate indicates that more than half of the positive vehicle-model images did not produce an accepted vehicle detection under the final experimental conditions. This result shows that the implemented low-resolution image workflow remained sensitive to the test conditions.

Possible contributing factors include:

- small physical scale of the vehicle models;

TABLE XX: Overall YOLO26l Results on the Final ESP32-CAM Dataset

Metric	Result	Interpretation
Total real input images	146	Positive vehicle-model images captured through the implemented ESP32-CAM workflow.
OPEN decisions	66	Images containing at least one allowed YOLO26l detection above the confidence threshold.
CLOSE decisions	80	Images for which no allowed detection satisfied the server decision rule.
Positive-sample detection rate	45.21%	Proportion of confirmed positive images producing OPEN.
Miss rate	54.79%	Proportion of confirmed positive images producing CLOSE.
Average inference time	432.23 ms	Mean server-side YOLO26l inference duration.
Median inference time	415.76 ms	Middle server-side inference duration.
Minimum inference time	386.61 ms	Lowest recorded server-side inference duration.
Maximum inference time	837.82 ms	Highest recorded server-side inference duration.
Inference-time standard deviation	62.16 ms	Variation of recorded inference durations around the mean.
95th-percentile inference time	505.17 ms	Ninety-five percent of recorded inference values were at or below this duration.
Mean accepted confidence	72.63%	Mean confidence among the 66 accepted detections.
Median accepted confidence	74.30%	Median confidence among the accepted detections.
Minimum accepted confidence	45.16%	Lowest accepted confidence, slightly above the configured threshold.
Maximum accepted confidence	94.75%	Highest confidence among the accepted detections.

- low object-to-image ratio;
- camera distance and viewing angle;
- lighting variation;
- image blur or compression;
- partial visibility of the vehicle model;
- the selected 0.45 confidence threshold.

These factors are plausible technical explanations rather than individually isolated causes because the experiment did not separately control and measure each imaging condition.

D. YOLO26l Results by Direction

The 146 results were divided into 95 ENTRY requests and 51 EXIT requests. Table XXI presents the results for each direction.

The EXIT subset produced a detection rate of 49.02%, which was 5.86 percentage points higher than the ENTRY result of 43.16%. The average inference time was also slightly lower for EXIT.

However, the two subsets contained different numbers of requests and were not documented as controlled datasets with identical image composition, lighting, distance, and orientation. Therefore, the result does not establish that the EXIT camera or direction was inher-

ently more accurate than ENTRY. It only describes the observed distribution in the available logs.

The ENTRY subset also contained the overall maximum inference time of 837.82 ms. This value is an isolated server-side inference result and not the complete maximum time from sensor detection to physical barrier movement.

E. Accepted Vehicle Classes

Among the 66 OPEN decisions, YOLO26l classified 43 accepted detections as truck and 23 as car. No accepted bus or motorcycle detections were recorded in the final dataset.

The dominance of truck predictions indicates that the physical appearance or scale of the vehicle models often matched features associated with the truck class according to YOLO26l. However, the available evidence does not establish that all 43 truck and 23 car predictions were manually verified against class-level ground truth.

Therefore, these values are reported as accepted model classifications rather than class-classification accuracy.

The barrier-control workflow treats all four allowed vehicle classes equally. A truck or car prediction above the threshold produces the same OPEN decision. Consequently, class confusion between two allowed vehicle

TABLE XXI: YOLO26l Results by Gate Direction

Direction	Requests	OPEN	CLOSE	Detection Rate	Average Inference	Median Inference	Maximum Inference
ENTRY	95	41	54	43.16%	436.99 ms	415.92 ms	837.82 ms
EXIT	51	25	26	49.02%	423.36 ms	413.07 ms	657.38 ms

TABLE XXII: Accepted Vehicle Classes in the Final Dataset

Predicted Class	Accepted Detections	Percentage of OPEN Results
Truck	43	65.15%
Car	23	34.85%
Bus	0	0.00%
Motorcycle	0	0.00%

categories does not necessarily change the binary barrier result, although it affects the semantic accuracy of the stored detection record.

F. Accepted-Detection Confidence

The accepted detection-confidence values ranged from 45.16% to 94.75%. The minimum accepted value was close to the configured threshold of 45%, while the median accepted confidence was 74.30%.

The mean accepted confidence was 72.63%. This indicates that accepted detections were not concentrated only near the threshold. Nevertheless, confidence describes the model's internal certainty for a particular prediction and must not be interpreted as the probability that the overall system is correct.

A high-confidence detection may still have an incorrect class label, and a low-confidence positive image may produce CLOSE when no detection passes the configured threshold. Complete reliability evaluation would require manual image annotation and per-image comparison with ground truth.

G. YOLO26l Inference-Time Analysis

The final experiment recorded an average YOLO26l inference time of 432.23 ms and a median of 415.76 ms. The median was lower than the mean, indicating that several slower requests increased the average.

The maximum inference time was 837.82 ms, which was substantially higher than the 95th-percentile result of 505.17 ms. Therefore, the maximum represented an uncommon high-duration request rather than typical inference behavior.

The recorded standard deviation of 62.16 ms shows measurable variation between requests. Potential sources include:

- image content and number of candidate detections;

- server resource availability;
- model warm-up effects;
- memory allocation;
- other processes running on the server;
- image decoding and preprocessing behavior.

The recorded inference time is only the YOLO processing component. It does not include every part of the system workflow.

The complete response path contains:

```

Ultrasonic detection
-> Slave processing
-> Master polling
-> Capture command
-> ESP32-CAM image capture
-> JPEG upload
-> YOLO inference
-> HTTP response return
-> UART propagation
-> I2C propagation
-> Master validation
-> Servo command
-> Servo movement

```

Therefore, the average inference time of 432.23 ms must not be reported as the average gate-opening time.

H. Comparison with the Historical YOLOv8n Configuration

YOLOv8n was used during an earlier development phase. The historical YOLOv8n run contained 46 image requests, of which 37 produced OPEN and 9 produced CLOSE. The average inference time was 93.42 ms.

To evaluate the final model using the previous test source, YOLO26l was rerun using the old vehicle-model test-case image collection. This comparison generated 52 YOLO26l result logs.

Table XXIII presents the available comparison.

The observed positive-sample detection-rate difference was:

$$\Delta R_{\text{detect}} = 94.23\% - 80.43\% = 13.80 \text{ percentage points.} \quad (11)$$

The observed miss-rate difference was:

TABLE XXIII: Historical YOLOv8n and YOLO26l Same-Source Comparison

Metric	Historical YOLOv8n Run	YOLO26l Comparison Run
Processed result logs	46	52
OPEN decisions	37	49
CLOSE decisions	9	3
Positive-sample detection rate	80.43%	94.23%
Miss rate	19.57%	5.77%
Average inference time	93.42 ms	421.39 ms
Median inference time	82.84 ms	403.47 ms
Minimum inference time	Not retained in the verified baseline	300.30 ms
Maximum inference time	326.67 ms	748.46 ms
Inference-time standard deviation	Not retained in the verified baseline	84.79 ms
95th-percentile inference time	Not retained in the verified baseline	590.43 ms
Mean accepted confidence	Not retained in the verified baseline	88.49%
Median accepted confidence	Not retained in the verified baseline	92.48%

$$\Delta R_{\text{miss}} = 5.77\% - 19.57\% = -13.80 \text{ percentage points.} \quad (12)$$

Therefore, YOLO26l produced fewer CLOSE results than YOLOv8n on the reused test-case image source.

However, the average inference-time ratio was:

$$\frac{421.39}{93.42} \approx 4.51. \quad (13)$$

YOLO26l therefore required approximately 4.51 times the average inference time of the historical YOLOv8n run.

The comparison demonstrates a practical trade-off:

- YOLOv8n provided substantially lower inference time;
- YOLO26l produced a higher positive-sample detection rate on the reused low-resolution vehicle-model test collection;
- YOLO26l required greater backend processing time and computational cost.

The result supports the selection of YOLO26l when detection consistency on the reused low-resolution test images is prioritized over inference speed.

However, the comparison is not a complete one-to-one paired benchmark. The available result counts differ:

YOLOv8n historical logs: 46
YOLO26l comparison logs: 52

The archive does not provide one matched YOLOv8n result and one matched YOLO26l result for every identical file. Therefore, the comparison must be described as a same-source practical comparison rather than a strict paired statistical experiment.

A complete paired benchmark would require:

- 1) the exact same image list for both models;
- 2) identical image identifiers;
- 3) identical confidence thresholds;

- 4) identical inference image sizes;
- 5) identical server hardware and runtime conditions;
- 6) manually verified ground-truth labels;
- 7) per-image detection and timing comparison.

I. Interpretation of the Two YOLO26l Datasets

The final 146-image dataset produced a positive-sample detection rate of 45.21%, whereas the reused old-image comparison set produced 94.23%.

These results must not be combined into a single average because they represent different image collections and experimental contexts. The substantial difference indicates that YOLO26l performance was strongly dependent on the content and capture conditions of the evaluated image set.

The 52-result comparison set was derived from the previous test-case source, while the 146-result final set represented the final ESP32-CAM experiment. Differences may include:

- camera placement;
- object distance;
- physical vehicle-model size;
- orientation;
- background;
- lighting;
- image sharpness;
- degree of vehicle visibility.

The available logs do not isolate these variables individually. Therefore, the results demonstrate dataset sensitivity but do not prove that any single factor caused the lower 45.21% result.

The comparison also demonstrates why a favorable result on one practical test collection cannot be generalized to all parking environments. The final model should be evaluated using a broader labeled dataset containing multiple positive and negative conditions.

J. Master-Slave Communication Results

The Arduino Master communicated with the ENTRY and EXIT Slaves through I2C. The ENTRY Slave used address 0×08 , while the EXIT Slave used address 0×09 .

Table XXIV summarizes the observed communication behavior.

The available communication evidence does not retain the exact number of polling cycles, expected responses, valid responses, or failed responses. Therefore, a precise communication-success percentage is not reported.

The result should be stated as:

Communication succeeded during all observed tests while both Slave modules were active.

It should not be generalized as 100% overall system reliability.

The inactive-Slave tests reveal a major architectural dependency. Although the modular design separates ENTRY and EXIT hardware, the current Master implementation requires both Slaves to remain active for normal operation.

A more fault-tolerant implementation should:

- 1) maintain separate health states for ENTRY and EXIT;
- 2) apply independent communication timeouts;
- 3) mark only the unresponsive module as unavailable;
- 4) allow the healthy direction to continue operating where safe;
- 5) display the failed module on the LCD and dashboard;
- 6) recover the module automatically when valid communication resumes.

K. Functional Behavior Summary

Table XXV summarizes the functional behaviors supported by the available implementation and evidence. No pass percentage or trial count is assigned where the raw count was not retained.

The table distinguishes verified server behavior, observed embedded behavior, and implemented fail-close logic from fully quantified end-to-end testing.

L. End-to-End Response-Time Analysis

End-to-end response time includes all stages between physical object detection and barrier movement.

The complete response time may be represented as:

$$T_{\text{end-to-end}} = T_{\text{sensor}} + T_{\text{poll}} + T_{\text{slave}} + T_{\text{capture}} + T_{\text{upload}} + T_{\text{inference}} + T_{\text{return}} + T_{\text{UART}} + T_{\text{I2C}} + T_{\text{master}} + T_{\text{servo}} \quad (14)$$

The terms are defined as follows:

- T_{sensor} : ultrasonic measurement and detection time;
- T_{poll} : waiting time before the Master observes the Slave state;
- T_{slave} : Slave command-processing time;
- T_{capture} : ESP32-CAM JPEG acquisition time;
- T_{upload} : HTTP image-upload time;
- $T_{\text{inference}}$: YOLO26l inference time;
- T_{return} : HTTP response-return time;
- T_{UART} : camera-to-Slave result transmission time;
- T_{I2C} : Slave-to-Master result transmission time;
- T_{master} : Master validation and command-processing time;
- T_{servo} : physical servo-movement time.

The following statement applies to the completed evaluation:

End-to-end gate response time was not quantitatively measured because the current prototype did not include synchronized timestamp logging across the Arduino Slave, ESP32-CAM, server, and Arduino Master. Therefore, only the YOLO26l inference time obtained from the server logs is reported.

The YOLO26l average inference time of 432.23 ms represents only $T_{\text{inference}}$. It excludes sensor polling, image capture, network transfer, decision propagation, and servo movement.

The configured timeouts also do not provide a measured response time. They specify maximum waiting boundaries at different software layers and may overlap.

M. Discussion

1) *Effectiveness of the Hybrid Architecture*: The hybrid architecture was designed to address the limitation of a sensor-only parking gate. The HC-SR04 sensor determines whether an object is located near the gate, while the ESP32-CAM and YOLO26l server provide an additional vehicle-verification stage.

This architecture offers two main advantages:

- 1) the camera workflow is activated only after an object is detected;
- 2) the barrier does not open directly from the ultrasonic distance threshold.

The sensor therefore reduces unnecessary image requests, while the server provides more semantic information than the sensor alone.

However, the final 45.21% positive-sample detection rate shows that adding a camera and large YOLO model does not automatically produce reliable verification. The effectiveness of the verification layer remains dependent on the quality and composition of the captured image.

TABLE XXIV: Observed Master–Slave Communication Results

Test Condition	Observed Result	Interpretation
Both Slaves active	Communication and normal Master operation succeeded during all observed tests.	Normal two-module operating condition was supported.
ENTRY Slave inactive	The Master stopped normal operation.	The system did not isolate the ENTRY module failure.
EXIT Slave inactive	The Master stopped normal operation.	The system did not isolate the EXIT module failure.
Both Slaves inactive	The Master stopped normal operation.	No normal directional operation remained available.
Master polling interval	Configured at 1000 ms.	Scheduling parameter, not measured I2C latency.

TABLE XXV: Functional Behavior Summary

ID	Scenario	Observed or Implemented Behavior	Evidence Limitation
F01	Both Slaves active	Master polling and normal communication proceeded during observed tests.	Exact polling-cycle count was not retained.
F02	ENTRY Slave inactive	Master stopped normal operation.	Fault-recovery duration was not measured.
F03	EXIT Slave inactive	Master stopped normal operation.	Fault-recovery duration was not measured.
F04	Both Slaves inactive	Master stopped normal operation.	No degraded operating mode was implemented.
F05	Allowed YOLO vehicle detection	Server returned OPEN when an allowed class satisfied the confidence threshold.	Server result alone does not confirm physical barrier movement.
F06	No accepted YOLO detection	Server returned CLOSE.	A CLOSE result does not prove that the image contained no vehicle.
F07	Invalid or malformed server result	ESP32-CAM decision logic treats non-OPEN output as CLOSE.	Repeated trial count was not retained.
F08	HTTP or server failure	Implemented policy produces a fail-close decision.	Complete repeated end-to-end failure trials were not recorded.
F09	Previous result at new-cycle start	Control design requires the previous result to be cleared or rejected.	Complete stale-result stress testing was not recorded.
F10	Object remains in detection area	Workflow design uses a wait-for-clear state before the next complete cycle.	Repeated object-clear trial count was not retained.

2) *Model Selection Trade-Off*: YOLO26l was selected because it produced more accepted detections than the historical YOLOv8n configuration on the reused low-resolution test-case source.

The practical trade-off was substantial:

- positive-sample detection rate increased by 13.80 percentage points;
- average inference time increased from 93.42 ms to 421.39 ms;
- YOLO26l required approximately 4.51 times the average inference duration.

For a small parking prototype, an inference duration near 400–500 ms may remain technically usable when the complete network and actuator workflow is stable. However, the complete gate response was not measured, so no claim can be made regarding the final user waiting time.

A smaller model may remain preferable when low processing delay is the primary requirement. A larger model may be preferable when improved detection consistency on difficult low-resolution images is more important.

The final selection therefore represents a project-specific trade-off rather than universal model superiority.

3) *Influence of Image Conditions*: The difference between the 45.21% final-dataset result and the 94.23% old-image comparison result demonstrates that the image collection has a major effect on observed performance.

The physical vehicle-model experiment differs from a conventional full-size vehicle dataset. The model vehicles occupy a smaller image region, and their proportions, textures, backgrounds, and camera distances may differ from the images used to train general-purpose YOLO models.

Improving the image-verification layer may require:

- moving the camera closer to the detection region;
- ensuring that the vehicle occupies a larger portion of the frame;
- stabilizing illumination;
- reducing motion blur;
- using a consistent background;
- adjusting the camera angle;
- collecting local training images;

- fine-tuning the model using the actual prototype environment;
- evaluating multiple confidence thresholds.

Lowering the confidence threshold could increase the number of OPEN results but may also increase false-positive decisions. A negative-sample dataset is required before this trade-off can be evaluated safely.

4) *Fail-Close Behavior*: The fail-close design is appropriate for preventing uncertain conditions from opening the barrier. Camera failure, Wi-Fi failure, server timeout, invalid UART data, and malformed responses are treated as closed decisions.

This policy improves conservative control behavior but reduces availability. A temporary network or camera failure prevents a valid vehicle from opening the barrier even when the ultrasonic sensor detects it correctly.

The system therefore prioritizes avoiding unsupported barrier opening over maintaining continuous service. Future implementations may add a controlled operator-review or manual recovery mode, but such a mode was not included in the current final scope.

5) *Master-Slave Dependency*: The I2C architecture successfully distinguished the ENTRY and EXIT modules when both Slaves were active. However, the Master stopped normal operation when either Slave became inactive.

This result shows that physical modularity does not automatically provide software fault isolation. Although each direction uses separate hardware, the Master workflow currently depends on the complete two-Slave communication sequence.

Independent health management would improve availability and allow one direction to continue operating when the other direction fails. Such behavior must still preserve the shared-barrier safety rules.

6) *Database and Dashboard Interpretation*: The database and dashboard provide a foundation for storing and presenting detection events. The available records can include module direction, decision, confidence, predicted class, timestamp, and relative image paths.

However, a server-side detection record does not prove that the Arduino Master opened the physical barrier. The current backend does not receive complete synchronized telemetry for every Master state, servo command, and object-clear event.

Therefore, dashboard records should be interpreted as backend detection evidence rather than complete physical gate audit records. A future telemetry channel from the Master to the backend would improve event traceability.

7) *Generalizability*: The experiment used physical vehicle models in a small laboratory prototype. The results cannot be generalized directly to:

- full-size cars or motorcycles;
- outdoor parking gates;
- night-time conditions;
- rain or weather exposure;
- high vehicle speed;
- industrial barrier mechanisms;
- congested Wi-Fi networks;
- long-duration unattended operation.

The prototype demonstrates the communication and control concept. Additional datasets, environmental tests, safety mechanisms, and hardware validation are required before practical deployment.

N. Results Summary

The final YOLO26l experiment processed 146 real ESP32-CAM images of physical vehicle models. The server produced 66 OPEN decisions and 80 CLOSE decisions, resulting in a positive-sample detection rate of 45.21% and a miss rate of 54.79%.

The mean inference time was 432.23 ms, while the median was 415.76 ms and the maximum was 837.82 ms. Accepted detections consisted of 43 truck predictions and 23 car predictions.

The same-source practical comparison showed that YOLO26l produced a higher positive-sample detection rate than the historical YOLOv8n configuration, but required approximately 4.51 times the average inference duration.

Master-Slave communication succeeded during all observed normal tests when both Slaves were active. The Master stopped normal operation when either or both Slaves became inactive, demonstrating that the current implementation does not support fault-isolated operation.

The evaluation supports the feasibility of the modular control architecture, but it also demonstrates limitations in image-detection consistency, processing cost, fault tolerance, evidence completeness, and end-to-end timing instrumentation.

VII. OBSERVED LIMITATIONS

The implemented prototype demonstrated the feasibility of combining ultrasonic sensing, ESP32-CAM image acquisition, server-side YOLO26l processing, Arduino Master-Slave communication, and servo-based barrier control. However, the experimental evidence also revealed several limitations that affect the interpretation, reliability, and generalizability of the reported results.

This section distinguishes limitations directly supported by implementation evidence from possible design concerns that were not quantitatively tested. The reported limitations therefore focus on the available image logs,

communication observations, power behavior, integration evidence, and experimental scope.

A. Positive-Only Vehicle-Detection Dataset

The principal YOLO26l experiment used 146 real ESP32-CAM images of physical vehicle models. According to the experimental description, every image contained a vehicle model. The dataset therefore contained positive vehicle samples only.

This dataset structure supports the calculation of:

- positive-sample vehicle-detection rate;
- positive-sample miss rate;
- number of OPEN decisions;
- number of CLOSE decisions;
- accepted predicted vehicle classes;
- accepted confidence-score statistics;
- server-side inference-time statistics.

However, the dataset does not contain confirmed negative images such as empty scenes, people, boxes, walls, or other non-vehicle objects. Consequently, the following metrics cannot be calculated from the available evidence:

- complete classification accuracy;
- precision;
- specificity;
- false-positive rate;
- F1-score;
- complete confusion matrix.

A positive-sample detection rate is not equivalent to overall model accuracy. Overall accuracy requires both positive and negative samples with manually verified ground-truth labels.

A more complete future dataset should contain:

- 1) vehicle images that should produce OPEN;
- 2) non-vehicle images that should produce CLOSE;
- 3) empty-scene images;
- 4) partially visible vehicles;
- 5) different vehicle orientations;
- 6) multiple lighting conditions;
- 7) multiple camera distances;
- 8) manually assigned ground-truth labels.

The image list should be fixed before evaluation so that every model is tested using exactly the same inputs.

B. High Miss Rate in the Final YOLO26l Experiment

YOLO26l detected an allowed vehicle class in 66 of the 146 positive vehicle-model images. The remaining 80 images produced CLOSE. The resulting positive-sample detection rate was 45.21%, while the miss rate was 54.79%.

The miss rate indicates that more than half of the confirmed positive samples did not satisfy the server decision rule. Therefore, the implemented image-verification layer cannot be described as highly accurate or fully reliable under the final experimental conditions.

The decision rule required at least one of the following classes to be detected with confidence greater than or equal to 0.45:

```
car
motorcycle
bus
truck
```

The observed misses may have been influenced by several image-related factors:

- physical vehicle models occupying a small part of the image;
- camera distance;
- camera angle;
- lighting variation;
- background complexity;
- partial vehicle visibility;
- image compression;
- motion blur;
- general-purpose model training data differing from the prototype scene.

These factors were not varied independently in controlled experiments. Therefore, they should be treated as plausible contributing conditions rather than confirmed individual causes.

The result demonstrates that the use of a larger YOLO model alone does not guarantee reliable detection when the captured image does not provide sufficient visual information. Improvements may require changes to both the camera setup and the model.

Potential improvements include:

- positioning the camera closer to the vehicle path;
- increasing the proportion of the image occupied by the vehicle;
- using stable and controlled lighting;
- reducing background variation;
- using a fixed capture position;
- collecting local training data;
- fine-tuning the model for physical vehicle models;
- testing multiple confidence thresholds;
- evaluating alternative image resolutions.

A lower confidence threshold may increase the number of OPEN results, but it may also increase false-positive decisions. This trade-off cannot be evaluated correctly until a negative-sample dataset is added.

C. Accepted Class Labels Were Not Fully Ground-Truth Verified

Among the 66 accepted detections, YOLO26l classified 43 results as truck and 23 as car. No accepted motorcycle or bus detections were present in the final dataset.

These values represent the model output classes. They do not prove that all class assignments were semantically correct because the available experiment did not retain a manually verified class label for every image.

For the binary barrier decision, both car and truck belong to the allowed vehicle-class set. Therefore, confusion between these two allowed classes may still produce the same OPEN decision.

However, class confusion affects:

- the semantic correctness of database records;
- the accuracy of dashboard class displays;
- future class-specific parking rules;
- any future analytical summary by vehicle category.

A future evaluation should include a manually labeled class for every image and report a class-level confusion matrix.

D. End-to-End Gate Response Time Was Not Measured

The complete system response begins when an object enters the ultrasonic detection region and ends when the SG90 servo reaches the required barrier position.

The complete timing path includes:

```

Ultrasonic detection
-> Slave processing
-> Master polling
-> Capture request
-> UART command
-> ESP32-CAM image capture
-> JPEG upload
-> YOLO26l inference
-> HTTP decision return
-> UART result propagation
-> I2C result propagation
-> Master validation
-> Servo command
-> Servo movement

```

The current prototype did not include synchronized timestamp logging across the Arduino Slave, ESP32-CAM, FastAPI server, Arduino Master, and servo-control stages.

Therefore, the required report statement is:

End-to-end gate response time was not quantitatively measured because the current prototype did not include synchronized timestamp

logging across the Arduino Slave, ESP32-CAM, server, and Arduino Master. Therefore, only the YOLO26l inference time obtained from the server logs is reported.

The average YOLO26l inference time of 432.23 ms represents only the model-processing component. It does not include sensor polling, image capture, image upload, response propagation, Master processing, or physical servo movement.

Similarly, the following values are software configuration parameters rather than measured response times:

- Master polling interval: 1000 ms;
- ESP32-CAM HTTP timeout: approximately 3500 ms;
- Slave capture timeout: approximately 6000 ms;
- Master capture timeout: approximately 7500–8000 ms;
- minimum barrier-open time: approximately 3000 ms.

A future timing experiment should define a common event identifier and record synchronized timestamps for:

- 1) first valid ultrasonic detection;
- 2) Master capture request;
- 3) Slave UART transmission;
- 4) ESP32-CAM capture start;
- 5) JPEG capture completion;
- 6) HTTP upload start;
- 7) server request receipt;
- 8) YOLO inference start and completion;
- 9) HTTP response transmission;
- 10) Slave result receipt;
- 11) Master result receipt;
- 12) servo command;
- 13) barrier movement completion.

The experiment should then report the number of trials, mean, median, minimum, maximum, and standard deviation.

E. Master Dependency on Both Slave Modules

The Arduino Master operated normally during the observed tests when both the ENTRY and EXIT Slaves were active and responsive.

However, the observed behavior changed when a Slave became inactive:

- when the ENTRY Slave was inactive, the Master stopped normal operation;
- when the EXIT Slave was inactive, the Master stopped normal operation;
- when both Slaves were inactive, the Master stopped normal operation.

This behavior shows that the current implementation does not provide fault isolation. Although the physical architecture is modular, the Master software depends on successful communication with both directional modules.

The system therefore cannot currently continue using the healthy gate direction when the other direction fails.

This limitation affects system availability. A failure in one of the following components may stop the complete Master workflow:

- one Arduino Slave;
- one Slave power connection;
- one I2C address configuration;
- one shared I2C connection;
- one local module state.

A future implementation should maintain independent module-health states such as:

```
ENTRY_AVAILABLE
ENTRY_UNAVAILABLE
EXIT_AVAILABLE
EXIT_UNAVAILABLE
```

The Master should then:

- 1) apply an independent timeout to each Slave;
- 2) isolate the unresponsive module;
- 3) continue polling the healthy module;
- 4) display the failed direction on the LCD;
- 5) report the failure to the dashboard where telemetry is available;
- 6) restore the module automatically when communication resumes.

Any degraded operating mode must still preserve exclusive Master control and fail-close behavior.

F. Communication Success Rate Was Not Fully Quantified

Communication succeeded during all observed normal tests when both Slaves were active. However, the raw communication-test evidence did not retain:

- total polling-cycle count;
- expected response count;
- valid ENTRY response count;
- valid EXIT response count;
- malformed response count;
- timeout count;
- retransmission count.

Therefore, the report does not claim a precise communication-success percentage.

The valid statement is:

Master-Slave communication succeeded during all observed tests in which both Slave modules were active and responsive.

The result must not be described as:

- 100% overall system reliability;
- fault-tolerant communication;
- reliable operation under all conditions;
- guaranteed long-duration I2C stability.

A future communication experiment should calculate:

$$R_{\text{communication}} = \frac{N_{\text{valid responses}}}{N_{\text{expected responses}}} \times 100\%. \quad (15)$$

The experiment should separately report normal operation, one-Slave failure, both-Slave failure, and recovery conditions.

G. Power Stability Affected ESP32-CAM Operation

Power stability affected camera operation during prototype integration. The ESP32-CAM operated correctly when powered using a laptop USB connection, but it was unstable through the previous adapter and voltage-conversion path.

This observation indicates that nominal adapter power alone is not sufficient to evaluate supply suitability. The ESP32-CAM requires a stable 5 V supply with sufficient transient-current capacity during:

- camera initialization;
- frame-buffer allocation;
- image capture;
- Wi-Fi connection;
- HTTP transmission.

The SG90 servo also creates short current changes during movement. When the camera and servo share an inadequate supply path, voltage drop may cause:

- ESP32-CAM restart;
- Wi-Fi disconnection;
- camera initialization failure;
- failed HTTP upload;
- incomplete UART result;
- unstable Arduino behavior.

The final design specifies a regulated 5 V/3 A supply with a common ground. However, the complete power system was not evaluated using current probes, oscilloscope measurements, voltage-drop logging, or long-duration load testing.

Therefore, the report can state that power instability was observed, but it cannot quantify:

- peak ESP32-CAM current;
- peak servo current;
- minimum supply voltage;

- voltage-ripple amplitude;
- brownout frequency;
- power efficiency.

A future power evaluation should measure the voltage directly at the ESP32-CAM and servo terminals during Wi-Fi transmission and barrier movement.

H. Complete End-to-End Workflow Was Not Repeated 146 Times

The 146-result YOLO26l dataset verifies server-side processing of real ESP32-CAM images. However, the complete hardware workflow was not documented through the same 146 repeated trials.

The server-side experiment confirms:

- image receipt;
- model inference;
- vehicle-class filtering;
- OPEN or CLOSE generation;
- inference-time logging.

It does not independently confirm that every request completed the full path:

```
Ultrasonic sensor
-> Arduino Slave
-> Arduino Master
-> Arduino Slave
-> ESP32-CAM
-> FastAPI
-> ESP32-CAM
-> Arduino Slave
-> Arduino Master
-> SG90 servo
```

Therefore, the 146 server requests must not be described as 146 successful barrier-control cycles.

A complete end-to-end reliability test should record, for every trial:

- 1) ultrasonic trigger result;
- 2) capture command delivery;
- 3) ESP32-CAM image success;
- 4) HTTP upload result;
- 5) YOLO decision;
- 6) UART decision delivery;
- 7) I2C result delivery;
- 8) Master validation;
- 9) servo response;
- 10) final object-clear state.

The report should then calculate a complete workflow success rate only when the raw numerator and denominator are available.

I. Dashboard and Database Evidence Was Limited

The SQL Server schema and Java dashboard were implemented to store and display parking information. Available fields include:

- gate direction;
- decision;
- confidence value;
- predicted class;
- raw image path;
- annotated image path;
- event timestamp;
- occupancy information;
- gate state.

However, the database and dashboard functions were not supported by sufficient repeated test cases, screenshots, timing records, and record-validation evidence.

Consequently, the report does not claim:

- perfect database-record completeness;
- measured dashboard-update latency;
- verified occupancy accuracy;
- complete synchronization between server decisions and physical barrier states;
- production-level dashboard availability;
- production-level dashboard security;
- guaranteed image-serving reliability.

A server-side detection record confirms that the back-end processed an image. It does not necessarily prove that the Arduino Master opened the barrier or that the vehicle completed a passage.

A more complete monitoring architecture should receive telemetry directly from the Arduino Master, including:

- active direction;
- capture-cycle identifier;
- final validated decision;
- servo command;
- barrier state;
- object-clear event;
- embedded timeout or error state.

This telemetry would allow the dashboard to distinguish between an AI decision and the final physical control result.

J. Limited Environmental Evaluation

The prototype was tested in a small laboratory environment. It was not evaluated under:

- long-duration continuous operation;
- unstable Wi-Fi;
- congested networks;
- high packet loss;

- broad lighting variation;
- night-time operation;
- outdoor weather exposure;
- rain or moisture;
- high ambient temperature;
- full-size vehicle distances;
- moving full-size vehicles;
- simultaneous high-volume ENTRY and EXIT events.

As a result, the report does not provide measured values for:

- system uptime;
- mean time between failures;
- Wi-Fi reconnection time;
- packet-loss rate;
- outdoor detection rate;
- low-light detection rate;
- long-duration memory stability;
- thermal behavior.

The existing results apply only to the implemented prototype and recorded experimental conditions.

K. Small-Scale Prototype Limitation

The physical system uses vehicle models and an SG90 servo barrier. This configuration is suitable for demonstrating the control sequence but is not equivalent to a real parking installation.

The prototype was not evaluated with:

- full-size vehicles;
- industrial barrier motors;
- barrier safety loops;
- emergency-stop systems;
- anti-crush protection;
- weatherproof enclosures;
- certified power supplies;
- industrial communication interfaces;
- legal access-control requirements.

The SG90 servo provides limited torque and is intended for small prototype mechanisms. It cannot be used as evidence that the same controller can directly operate a production barrier.

The current system should therefore be described as a proof-of-concept prototype rather than a deployment-ready parking system.

L. Same-Source Model Comparison Was Not Fully Paired

YOLO26l was rerun using the old test-case image source previously used for YOLOv8n. The historical

YOLOv8n run contained 46 result logs, while the YOLO26l rerun contained 52 result logs.

The observed comparison was:

- YOLOv8n positive-sample detection rate: 80.43%;
- YOLO26l positive-sample detection rate: 94.23%;
- observed difference: 13.80 percentage points;
- YOLOv8n average inference time: 93.42 ms;
- YOLO26l average inference time: 421.39 ms.

The comparison supports the observation that YOLO26l produced more accepted detections on the reused test-case source. However, it is not a complete paired benchmark because the available result counts differ and matching outputs were not retained for every image.

The comparison does not establish universal superiority of YOLO26l. It applies only to the observed low-resolution vehicle-model test source.

A strict paired comparison should require:

- 1) identical input file list;
- 2) identical image identifiers;
- 3) one result from each model for every image;
- 4) identical confidence thresholds;
- 5) identical image sizes;
- 6) identical server hardware;
- 7) identical runtime conditions;
- 8) manually verified ground truth;
- 9) per-image inference-time comparison.

M. Model Performance Was Dataset-Dependent

The 146-image final dataset produced a detection rate of 45.21%, whereas the old-image YOLO26l comparison run produced 94.23%.

This difference indicates that the observed model performance depended strongly on the evaluated image collection. The two results should not be merged or averaged.

The difference may reflect changes in:

- camera configuration;
- object scale;
- camera distance;
- image orientation;
- background;
- lighting;
- focus;
- degree of vehicle visibility.

Because these variables were not independently controlled, the experiment does not identify one confirmed cause for the performance difference.

This limitation demonstrates the need for a standardized local benchmark dataset and a controlled capture procedure.

N. License-Plate Recognition Was Not Implemented

The current backend performs vehicle-class detection. It does not implement:

- license-plate localization;
- optical character recognition;
- plate-text validation;
- registered-vehicle comparison;
- entry–exit plate matching.

Although license-plate recognition appears in some reviewed literature, it is outside the completed project scope and may only be presented as a possible future extension.

The OPEN decision therefore means:

An allowed vehicle class was detected above the configured confidence threshold.

It does not mean:

The vehicle was identified or authorized by license plate.

O. No Quantitative HC-SR04 Accuracy Evaluation

The HC-SR04 sensors were used as object-presence triggers. However, the project did not complete a controlled experiment comparing sensor measurements with ground-truth distances across multiple trials.

Therefore, the report does not provide:

- mean distance error;
- maximum distance error;
- sensor precision;
- false-trigger rate;
- missed-trigger rate;
- environmental sensitivity.

The configured object-detection threshold is a design parameter rather than a measured accuracy result.

A future sensor experiment should use fixed reference distances and record repeated measurements under:

- different object materials;
- different object angles;
- different vehicle-model sizes;
- different temperatures;
- object-present and object-absent conditions.

P. Barrier-Safety Evaluation Was Not Performed

The system uses a minimum-open time and object-clear logic to reduce repeated or premature barrier movement. However, the prototype did not undergo a formal safety evaluation.

The system was not tested for:

- physical obstruction during closing;
- servo stall;

- barrier collision;
- sensor failure while the barrier is open;
- emergency stopping;
- human or vehicle safety compliance.

A fixed minimum-open timer does not guarantee safe vehicle passage. A production system would require additional independent safety sensors and certified barrier-control hardware.

Q. Design Concerns Not Quantitatively Evaluated

Some potential concerns were identified through architecture and code review but were not supported by sufficient direct experimental evidence.

These concerns include:

- limited UART message context;
- sequence-identifier robustness;
- result acknowledgement edge cases;
- priority behavior during simultaneous ENTRY and EXIT detections;
- absence of heartbeat monitoring.

These items should be documented as design limitations or future improvements rather than confirmed observed failures.

For example, the current scope does not implement a heartbeat mechanism. However, the absence of heartbeat should not automatically be described as an observed failure unless an experiment demonstrates that its absence caused a measurable problem.

Similarly, the communication design should reject stale or mismatched results, but a complete sequence-ID stress test was not retained. Therefore, sequence robustness remains an area for future validation.

R. Implications for Result Interpretation

The reported results must be interpreted within the following boundaries:

- 1) The 45.21% value is a positive-sample detection rate, not overall accuracy.
- 2) The 54.79% value is a miss rate on positive vehicle-model images.
- 3) Confidence scores are model outputs, not accuracy percentages.
- 4) The 432.23 ms average is YOLO26l inference time, not end-to-end gate response time.
- 5) The 146 server logs are not equivalent to 146 complete barrier cycles.
- 6) Communication success was observed under the normal two-Slave condition but was not quantified using retained polling counts.
- 7) Dashboard and database implementation evidence does not establish complete monitoring reliability.

- 8) Results obtained using vehicle models cannot be generalized to full-size parking environments.
- 9) YOLO26l performed better than YOLOv8n only in the observed same-source practical comparison.

These limitations do not invalidate the implemented prototype. They define the evidence boundary within which the system can be evaluated accurately.

S. Required Future Evaluation

A more complete future evaluation should include the following experimental groups:

- 1) A labeled positive and negative image dataset.
- 2) A strict one-to-one YOLOv8n and YOLO26l paired benchmark.
- 3) Controlled camera-distance and lighting tests.
- 4) HC-SR04 accuracy and false-trigger testing.
- 5) Complete synchronized end-to-end response-time measurement.
- 6) Repeated full-workflow barrier trials.
- 7) Independent ENTRY and EXIT communication reliability tests.
- 8) Single-Slave failure and recovery tests.
- 9) ESP32-CAM power and voltage-drop measurements.
- 10) Wi-Fi interruption and reconnection tests.
- 11) Database-event completeness validation.
- 12) Dashboard-update delay measurement.
- 13) Long-duration system-uptime testing.
- 14) Barrier object-clear and safety testing.

Each experiment should retain the raw logs, trial count, expected result, actual result, pass/fail result, timestamp, and associated evidence file.

T. Limitations Summary

The principal limitation of the AI evaluation is that the final 146-image dataset contained positive vehicle samples only and produced a 54.79% miss rate. Therefore, complete classification metrics could not be calculated, and image-verification reliability remained limited under the final capture conditions.

The principal embedded limitation is that the Arduino Master depends on both Slave modules. The system stops normal operation when either direction becomes inactive and does not provide degraded single-direction operation.

The principal integration limitation is that server-side image tests were not matched by the same number of complete end-to-end barrier trials. Consequently, server decisions cannot be treated as proof of complete physical-control success.

The principal measurement limitation is the absence of synchronized timestamps across all modules. Only server-side YOLO26l inference time was measured.

The principal deployment limitation is that the system was evaluated using physical vehicle models and a small servo barrier in a laboratory environment. Production operation, full-size vehicles, outdoor conditions, industrial safety, and long-duration reliability remain outside the completed evaluation.

VIII. CONCLUSION

This project designed, implemented, and evaluated a small-scale smart parking prototype that combines ultrasonic object-presence sensing, ESP32-CAM image acquisition, server-side vehicle detection, Arduino-based communication, servo-barrier control, database storage, and web-based monitoring. The final architecture separates the system into local control, directional sensing, camera communication, backend processing, and monitoring layers.

The embedded system uses one Arduino Uno R3 Master and two Arduino Uno R3 Slaves. The ENTRY Slave is assigned I2C address 0×08 , while the EXIT Slave is assigned address 0×09 . Each Slave manages one HC-SR04 ultrasonic sensor and one ESP32-CAM module. The Arduino Master coordinates the complete local workflow, validates the returned detection result, controls the SG90 servo barrier, and updates the parallel LCD1602 used for local debugging and status observation.

The implemented control structure applies a hybrid detection approach. Ultrasonic sensing is used as the initial object-presence trigger because it is inexpensive, simple to interface, and appropriate for low-cost embedded prototypes. However, the ultrasonic result does not directly authorize barrier opening because the sensor cannot determine whether the detected object is a vehicle. Instead, the corresponding ESP32-CAM captures a JPEG image and sends it through HTTP over Wi-Fi to the FastAPI backend.

The FastAPI server executes the final YOLO26l model stored in `yolo26l.pt`. The inference process uses an image size of 640 pixels, a confidence threshold of 0.45, and the allowed classes `car`, `motorcycle`, `bus`, and `truck`. The server returns `OPEN` only when at least one allowed vehicle class satisfies the configured confidence threshold. If no accepted detection is available, the server returns `CLOSE`.

The Arduino Master remains the only module permitted to control the SG90 servo. The barrier begins in the closed position and opens only after the Master

receives a valid `OPEN` result associated with the active capture cycle. Camera initialization failure, Wi-Fi failure, JPEG capture failure, HTTP timeout, server failure, malformed response, invalid UART data, stale result, or Master timeout keeps the barrier closed. This fail-close policy prevents uncertain communication or processing conditions from directly opening the barrier.

The final YOLO26l experiment processed 146 real ESP32-CAM images of physical vehicle models. The server produced 66 `OPEN` decisions and 80 `CLOSE` decisions. Because every test image contained a vehicle model, the experiment produced a positive-sample detection rate of 45.21% and a miss rate of 54.79%.

The average YOLO26l inference time was 432.23 ms, while the median inference time was 415.76 ms. The minimum recorded inference time was 386.61 ms, and the maximum was 837.82 ms. Among the accepted detections, 43 were classified as truck and 23 as car. The mean accepted confidence was 72.63%, with accepted values ranging from 45.16% to 94.75%.

These results demonstrate that the implemented backend can receive ESP32-CAM images, execute YOLO26l, filter vehicle classes, produce a binary barrier decision, and record the associated inference information. However, the positive-sample detection rate also demonstrates that the final image-verification process remained strongly affected by the prototype conditions. Vehicle-model scale, camera position, lighting, orientation, image compression, and the selected confidence threshold may all influence whether a valid vehicle detection is produced.

A separate same-source practical comparison was performed using the previous vehicle-model test-case image collection. The historical YOLOv8n run produced a positive-sample detection rate of 80.43%, while the YOLO26l rerun produced 94.23%. The observed improvement was 13.80 percentage points.

This improvement was accompanied by a substantial processing-time cost. The historical YOLOv8n average inference time was 93.42 ms, whereas the YOLO26l comparison run averaged 421.39 ms. Therefore, YOLO26l required approximately 4.51 times the average inference duration of YOLOv8n.

The comparison supports the selection of YOLO26l for the final prototype when detection consistency on the reused low-resolution test images is prioritized over server inference speed. However, it does not establish universal superiority because the available result counts differed and the comparison did not retain one-to-one matched results for every input image.

The Arduino Master-Slave communication structure successfully distinguished the `ENTRY` and `EXIT` mod-

ules during observed tests when both Slaves were active. The modular architecture provided clear separation between directional sensing and central barrier control. Nevertheless, the Master stopped normal operation when either one or both Slaves became inactive. The current implementation therefore does not provide fault isolation or continued single-direction operation.

The project also implemented SQL Server and Java web-dashboard components to support event persistence and monitoring. Available records can include gate direction, decision, predicted vehicle class, confidence value, timestamp, raw image path, annotated image path, gate state, and occupancy information. These components demonstrate the intended integration between the AI backend and a centralized monitoring interface.

However, the database and dashboard were not evaluated through sufficient repeated record-validation tests to claim complete event consistency, occupancy accuracy, dashboard-update delay, or production-level availability. In addition, a server-side detection record does not independently prove that the Arduino Master opened the physical barrier. Complete physical gate telemetry from the Master to the backend remains a future requirement.

The project achieved its primary architectural objective by implementing a complete logical path from ultrasonic detection to server-side vehicle verification and Master-controlled barrier actuation. It demonstrated the feasibility of combining low-cost embedded devices with a local AI server while maintaining clear control ownership and conservative failure behavior.

The evaluation also identified important limitations. The main image dataset contained positive vehicle samples only and therefore did not support complete accuracy, precision, specificity, false-positive rate, or F1-score calculations. End-to-end gate response time was not quantitatively measured because synchronized timestamps were not recorded across the Slave, ESP32-CAM, server, Master, and servo stages. Power stability affected ESP32-CAM operation during integration, and the complete barrier-control workflow was not repeated for all 146 server-side image requests.

The completed system should therefore be interpreted as a proof-of-concept prototype rather than a production-ready parking solution. It demonstrates the integration of sensing, image capture, AI processing, communication, barrier control, data storage, and dashboard monitoring, but it has not been validated for full-size vehicles, industrial barriers, outdoor conditions, long-duration operation, or production safety requirements.

A. Achievement of Project Objectives

The completed implementation achieved the following objectives:

- 1) Two HC-SR04 sensors were used to provide initial object-presence detection for the ENTRY and EXIT directions.
- 2) One Arduino Uno R3 Master and two Arduino Uno R3 Slaves were integrated through I2C using the fixed addresses 0x08 and 0x09.
- 3) Two ESP32-CAM modules were configured to capture JPEG images and communicate with the backend through Wi-Fi.
- 4) The final camera configuration used a 20 MHz external camera clock, VGA frame size, JPEG quality value 12, and one frame buffer.
- 5) The FastAPI backend was configured to execute the YOLO26l vehicle-detection model.
- 6) The server generated OPEN and CLOSE decisions using the defined vehicle-class and confidence rules.
- 7) The Arduino Master retained exclusive control of the SG90 servo barrier.
- 8) The system applied a fail-close policy for invalid or uncertain conditions.
- 9) SQL Server and Java dashboard components were developed to support event storage and monitoring.
- 10) The final YOLO26l model was evaluated using 146 retained result logs from real ESP32-CAM vehicle-model images.
- 11) The measured server-side inference statistics and observed Master-Slave communication behavior were documented without replacing unavailable metrics with estimated values.

Several intended evaluation objectives were not fully completed. HC-SR04 detection accuracy, complete workflow success rate, dashboard-update delay, occupancy accuracy, system uptime, and synchronized end-to-end gate response time remain unmeasured.

B. Future Work

Future work should first improve the image dataset and experimental methodology. A labeled dataset should be created using the actual ESP32-CAM modules and prototype environment. It should contain both positive and negative samples, including vehicles, non-vehicle objects, empty scenes, partial vehicles, multiple orientations, and different lighting conditions.

Each image should have a manually verified ground-truth label. This would support calculation of:

- true positives;
- true negatives;

- false positives;
- false negatives;
- accuracy;
- precision;
- recall;
- specificity;
- F1-score;
- confusion matrices.

A strict paired comparison between YOLOv8n and YOLO26l should use the exact same image files, confidence threshold, inference size, server hardware, and runtime conditions. The result should include one detection outcome and one inference time for each model and each image.

The camera environment should also be improved. The vehicle should occupy a larger portion of the frame, and the capture position should be standardized. Controlled illumination, fixed camera angle, stable focus, and reduced background variation may improve detection consistency.

Model fine-tuning should be considered using images collected from the actual prototype. A locally fine-tuned smaller model may provide a more effective balance between detection reliability and processing time than a general-purpose large model.

The end-to-end workflow should be instrumented with synchronized timestamps. A unique event or sequence identifier should be propagated through the Master, Slave, ESP32-CAM, backend, database, and dashboard. This would allow each stage of the complete response time to be measured independently.

Future timing records should include:

- first valid ultrasonic detection;
- Master capture request;
- Slave UART command;
- camera capture start and completion;
- HTTP upload start and completion;
- server request receipt;
- YOLO inference start and completion;
- result transmission;
- Slave result receipt;
- Master result validation;
- servo command;
- barrier movement completion.

The Arduino Master should be modified to support independent module-health management. If one Slave becomes unavailable, the Master should isolate that module, report its failure, and continue operating the healthy direction where safe.

A future communication protocol should include:

- event sequence identifier;

- module identity;
- capture status;
- result code;
- acknowledgement;
- error code;
- integrity validation.

The power system should be evaluated using direct voltage and current measurements. Voltage should be measured at the ESP32-CAM and servo terminals during camera initialization, Wi-Fi transmission, image upload, and servo movement. Appropriate decoupling, branch wiring, and regulated supply design should then be validated under simultaneous load.

The database should receive telemetry directly from the Arduino Master. Future records should distinguish between:

- server detection decision;
- Master-validated decision;
- servo command;
- physical barrier state;
- object-clear state;
- embedded timeout or error state.

The dashboard should be evaluated using repeated test cases that verify event completeness, image display, gate-state consistency, occupancy updates, and update delay.

License-plate recognition may be added as a future extension. Such an implementation would require plate localization, optical character recognition, text validation, vehicle registration records, and authorization rules. It should remain separate from the current vehicle-class detection claim.

For practical deployment, the SG90 prototype barrier would need to be replaced with an industrial barrier system. Independent safety sensors, obstruction detection, emergency stopping, weatherproof hardware, secure communication, user authentication, and production-grade power management would also be required.

C. Final Statement

The project demonstrates that a low-cost smart parking prototype can integrate ultrasonic detection, ESP32-CAM image capture, server-side YOLO processing, Arduino Master-Slave control, SQL Server storage, and web-based monitoring within one modular architecture.

The implementation confirms the feasibility of the control concept but also shows that reliable smart parking operation depends on more than the selected AI model. Image quality, power stability, communication state management, timeout coordination, event synchronization, actuator safety, and evidence-based evaluation are all critical to the final system behavior.

The completed prototype provides a technical foundation for future development. Its next stage should focus on improving local vehicle-detection performance, adding negative-sample evaluation, measuring complete gate-response time, isolating module failures, validating database and dashboard consistency, and testing the complete workflow under controlled repeated conditions.

REFERENCES

- [1] A. Z. M. T. Kabir, A. M. Mizan, P. K. Saha, M. S. Hasan, and M. Pramanik, "An IoT-based intelligent parking system for the unutilized parking area with real-time monitoring using mobile and web application," *Asian Journal of Convergence in Technology*, vol. 7, no. 2, pp. 107–113, 2021.
- [2] I. Jung, "An IoT-based smart parking management system," *International Journal of Computational Vision and Robotics*, vol. 10, no. 2, pp. 122–132, 2020.
- [3] S. Y. C. Hong, C. C. Kang, J. D. Tan, and M. Ariannejad, "Smart parking system using IoT sensors," *Journal of Engineering Technology and Applied Physics*, vol. 5, no. 1, pp. 63–69, 2023.
- [4] Y. Allbadi, J. N. Shehab, and M. M. Jasim, "The smart parking system using ultrasonic control sensors," *IOP Conference Series: Materials Science and Engineering*, vol. 1076, no. 1, p. 012064, 2021.
- [5] E. Saputra, N. P. IS, E. Fitriani, T. Ariyadi, and A. H. A. Tarecha, "Car parking monitoring system based on location plate detection using web camera," *Circuit: Jurnal Ilmiah Pendidikan Teknik Elektro*, vol. 10, no. 1, pp. 25–39, 2026.
- [6] G. P. C. P. da Luz, G. M. Sato, L. F. G. Gonzalez, and J. F. Borin, "Smart parking with pixel-wise ROI selection for vehicle detection using YOLOv8, YOLOv9, YOLOv10, and YOLOv11," 2024, version 2.
- [7] V. A. Kusuma, H. Arof, S. S. Suprpto, B. Suharto, R. A. Sinulingga, and F. Ama, "An internet of things-based touchless parking system using ESP32-CAM," *International Journal of Reconfigurable and Embedded Systems*, vol. 12, no. 3, pp. 329–335, 2023.